

netsim: Convergence-First Network Simulator

Design & Architecture

Simon Knight

March 2026

Last updated: March 18, 2026

Abstract. netsim is a tick-based network simulator written in Rust that bridges the gap between algorithmic configuration analysis and container-based network emulation. It provides protocol-accurate simulation of OSPF (v2/v3), IS-IS, BGP (including EVPN/VXLAN and L3VPN), MPLS/LDP, RSVP-TE, BFD, LACP, LLDP, RIPv2/RIPng, STP, VRRP v2/v3, DHCP Relay, IGMPv2, and PIM-SM with deterministic convergence detection at 486-device scale. This technical report documents the system's architecture, design rationale, data model, and usage. It is intended for engineers evaluating netsim for network validation, contributors to the codebase, and researchers interested in the convergence-first simulation model.

Contents

1	Introduction	1
2	Data Model	2
2.1	Devices	2
2.2	Wires and Interface QoS	3
3	Forwarding Architecture	3
3.1	RIB/FIB Separation	3
3.2	Dual-Index FIB	4
3.3	ECMP	5
4	Control Plane Routing	5
4.1	OSPF	6
4.2	BGP	6
4.3	RIP	7
5	Data Plane & Encapsulation	7
5.1	Hybrid Real-Sim Data Plane (TUN/TAP Bridge)	7
5.2	MPLS and RSVP-TE	7
5.3	SRv6 (Segment Routing over IPv6)	7
5.4	VXLAN	8
6	Infrastructure & Edge	8
6.1	BFD, LACP, LLDP	9
6.2	STP	9
6.3	VRRP	9
6.4	DHCP Relay	9
6.5	Multicast: IGMP, PIM-SM, and MFIB	9
7	Declarative Topology & Chaos Engineering	11
7.1	YAML Topology Format	11
7.2	Pattern-Based Selectors	11
7.3	Declarative Assertions	11
7.4	Failure Patterns	12
8	EVPN/VXLAN at Simulation Abstraction	13
8.1	Problem	13
8.2	Unified Router as L2+L3 Device	13
8.3	Bridge Domain and FDB	14
8.4	BGP EVPN Control Plane	15
8.5	VXLAN Encapsulation and Decapsulation	15
8.6	MAC Mobility	15
8.7	ARP Suppression	16
8.8	Snapshot-Then-Apply Synchronisation	16
8.9	EVPN Multi-Homing	16
9	CLI Tool & Interactive Daemon	17
9.1	CLI Subcommands	17
9.2	Operational Emulation via Vendor Skinning	17
9.3	Daemon Architecture	18
9.4	gRPC Service Interface	20
9.5	IOS-Style Command Tree	20
9.6	Ping and Traceroute	20
9.7	Diagnostic Tools: dig and iperf	21
9.8	Static Analysis: Path Tracing, Diffing, and Capacity	21
9.9	JSON Structured Output	21
9.10	Command Determinism	21
9.11	TUI Device Selector	21
9.12	Shell Completions & JSON Schema Emit	21
9.13	netsim up: Zero-Setup Interactive Sessions	22
9.14	Host Shorthand & Bulk Script Commands	22
9.15	Simulation Timeline & Event Log	23
9.16	Plan Preview Mode	24

9.17	Contextual Error Messages	24
9.18	CI/CD Output Formats	25
10	Deterministic Sandbox for Network AI and Automation	25
11	Python Bindings and the Deterministic AI Sandbox	26
11.1	AI Agent Integration Example	27
11.2	Architecture.	27
11.3	Topology Construction API	28
11.4	Simulation Control	28
11.5	Device State Inspection.	29
11.6	Event Scheduling.	29
11.7	Exception Hierarchy	30
11.8	Integration with pytest	30
12	Usage	30
12.1	Example 1: Simple Reachability Test	30
12.2	Example 2: OSPF Reconvergence After Link Failure	31
12.3	Example 3: BGP L3VPN with Post-Convergence Inspection.	31
12.4	Example 4: Chaos Engineering	31
12.5	Example 5: VRRP Gateway Redundancy	32
12.6	Example 6: DHCP Relay Across Subnets	32
12.7	Example 7: SRv6 Traffic Steering	32
12.8	Example 8: BGP Route Reflection.	32
12.9	Example 9: Multicast Group Membership	33
13	Example Topologies	33
13.1	Built-in Examples (netsim example)	33
13.2	Extended Examples	34
14	Case Study: Validating Cascading Protocol Failures (Facebook 2021 Outage)	35
14.1	Modeling the Scenario in netsim	35
14.2	The Cascading Failure State Machine.	36
14.3	Execution and CI/CD Impact	36
15	Case Study: Data Centre Fabric Validation (EVPN/VXLAN)	36
15.1	Modeling the Fabric in netsim	36
15.2	Execution and Validation Impact	37
16	Case Study: Service Provider L3VPN with Overlapping IP Space	37
16.1	Modeling VRFs and Route Targets	37
16.2	Execution and Validation Impact	38
17	Case Study: Automated Remediation via Python AI Agent	38
17.1	The Agent Control Loop	38
17.2	Execution and Validation Impact	39
18	Speculative Remediation: Monte Carlo Tree Search for Networks	39
18.1	The Multi-Universe Remediation Loop	39
18.2	Architectural Enablement.	39
19	Deterministic Blast Radius Explorer	39
20	The AI Network Doctor: LLM-Augmented Diagnostics	40
21	Digital Twin Ingestion: From Production to Simulation	40
22	System Architecture	41
22.1	Build Stage.	41
22.2	Run Stage: The Tick Loop.	41
22.3	Rust Ownership and Data Flow	42
22.4	Determinism Guarantees	42
22.5	Adaptive Parallelism	42
22.6	Output Stage	42
23	Convergence Detection	43
23.1	Problem	43
23.2	Design	43

23.3	Convergence as a Time Coordinate	45
24	Experimental Evaluation	45
24.1	E1: Ground Truth Fidelity via Differential Testing	45
24.2	E2: Determinism at Scale and Execution Speed	46
24.3	E3: Tick Sensitivity and Aliasing	46
24.4	E4: Scale Ceiling and Bottlenecks	46
24.5	E5: False Convergence and Metastability	46
25	Performance Architecture	47
25.1	Zero-Copy SoA Memory Architecture	47
25.2	Benchmark Infrastructure	47
25.3	Metrics	48
25.4	Scalability	48
25.5	Zero-Copy SoA Device Storage	48
25.6	BatchQueue and Priority Queuing	48
25.7	Phase Timing Instrumentation	49
25.8	Incremental FIB with Dirty Flags	49
25.9	Adaptive Parallelism	49
25.10	Execution Modes	49
25.11	Regression Testing	49
25.12	Control Plane Policing (CoPP)	50
25.13	End-to-End Hardening	50
26	Limitations	50
26.1	No TCP/UDP Transport Modelling	50
26.2	Idealised Device Behaviour	50
26.3	No Detailed Queuing Theory	50
26.4	Scale Ceiling	50
26.5	EVPN Synchronisation Fidelity	51
26.6	BFD Auto-Session Wiring	51
27	Future Work	51
28	Related Work	51
A	Assertion Type Reference	54
B	YAML Schema Quick Reference	54
C	CLI Command Quick Reference	55

Abstract

Network infrastructure relies heavily on automated CI/CD pipelines and, increasingly, AI-driven management agents. However, existing network emulation environments—based on virtual machines or containerised routing daemons—are fundamentally bottlenecked by OS scheduling jitter and high memory overhead, making them non-deterministic and too slow for rapid validation loops. We present *netstim*, a high-fidelity control plane simulator engineered explicitly for continuous integration and deterministic AI sandboxing. By employing a lock-free, zero-copy Structure-of-Arrays (SoA) tick engine written in Rust, *netstim* models 17 protocol families (including OSPF, BGP, EVPN/VXLAN, and MPLS) natively without underlying kernel dependencies. Routing convergence is elevated from a side effect of elapsed wall-clock time to a mathematically precise, addressable coordinate. This enables complex, cascading failure scenarios—such as hyperscaler BGP withdrawals triggered by physical layer faults—to be modeled, executed, and exhaustively asserted against in mere milliseconds. We detail the system’s declarative design, unified forwarding architecture, and Python bindings, demonstrating how mathematical determinism fundamentally changes the viability of software-engineering-grade testing in network operations.

1 Introduction

netstim is a network simulator that models routing protocol behaviour at packet granularity with deterministic, reproducible results. It accepts a single YAML file describing a network topology—devices, links, protocol configurations, traffic generators, failure patterns, and assertions—and simulates the network tick by tick until all routing protocols converge. It implements 17 protocol families spanning link-state routing (OSPF v2/v3, IS-IS), distance-vector routing (RIPv2/RIPng), path-vector routing (BGP-4 with EVPN), label switching (MPLS/LDP, RSVP-TE), L2 infrastructure (STP, LACP, LLDP, VXLAN), first-hop redundancy (VRRP v2/v3), address management (DHCP Relay), multicast (IGMPv2, PIM-SM), and fast failure detection (BFD). It replaces the need for

container-based emulation [1] in scenarios where determinism and speed matter more than vendor-specific NOS fidelity.

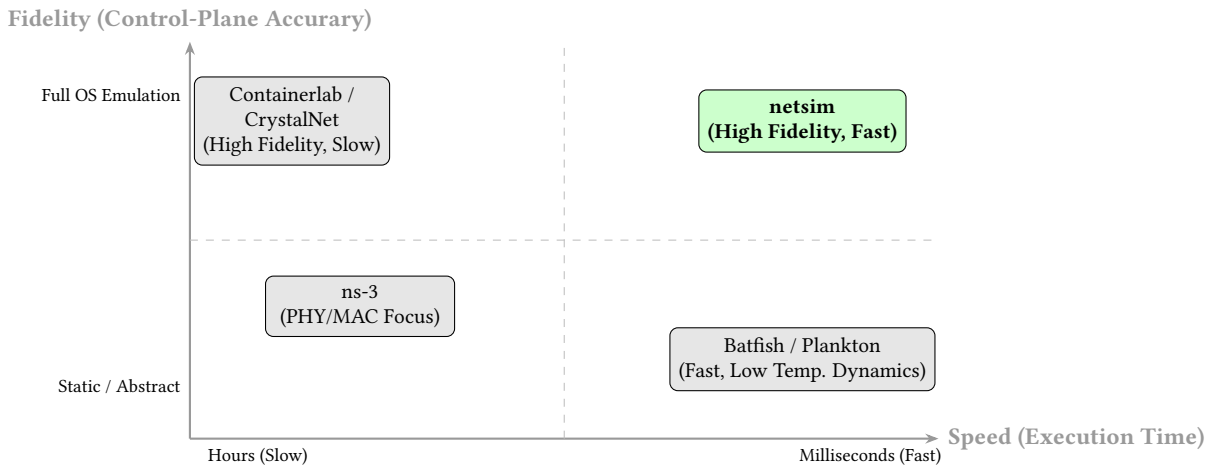


Figure 1. The Simulator Impedance Mismatch. `netsim` occupies the target quadrant for CI/CD pipelines: executing high-fidelity control-plane protocols (like OS emulation) but at deterministic, millisecond speeds (like static analysis).

The primary contribution of `netsim` is **Convergence as a Time Coordinate**. Routing convergence is not treated as a side effect of “running long enough,” but as a first-class, addressable time dimension. Two core design principles explain every subsequent decision in this document:

1. **Convergence is a first-class time coordinate.** Routing convergence is not a side effect of “running long enough.” It is a precisely defined predicate (the simultaneous stability of 11 protocol states) that is observable, addressable, and scriptable. Events, assertions, and chaos engineering all reference convergence as a time anchor (at: converged + 500ms).
2. **Determinism over fidelity.** `netsim` models RFC behaviour, not vendor-specific implementation quirks. All timers are integer ticks, all RNGs are seeded, and all protocol synchronisation uses snapshot-then-apply rather than per-message queuing. This trades the last mile of realism for bit-identical reproducibility across runs.

This document is intended for three audiences: network engineers evaluating `netsim` for pre-deployment validation in CI/CD pipelines, contributors to the codebase who need to understand architectural decisions before modifying the system, and researchers interested in the convergence-first simulation model and its implications for network testing.

Familiarity with IP routing protocols (OSPF, BGP) and basic Rust syntax is assumed. Experience with network emulation tools (Containerlab, ns-3) is helpful but not required.

Section 2 defines the core data model (devices, wires, interfaces). Section 3 covers the forwarding architecture (RIB/FIB, ECMP). Sections 4 to 6 survey all 17 protocol families (including RIP, STP, VRRP, DHCP Relay, and IGMP/PIM-SM multicast). Section 7 describes the YAML topology format and chaos engineering. Section 8 covers EVPN/VXLAN at simulation abstraction. Section 9 documents the CLI tool, interactive daemon, diagnostic commands (traceroute, dig, iperf), and path tracing engine. Section 12 provides end-to-end usage examples. Section 13 catalogues the 50+ example topologies shipped with `netsim` and their protocol coverage. Section 14 details a full real-world cascading failure case study. Section 22 presents the system architecture and the lock-free tick pipeline. Section 23 explains the convergence predicate in detail. Section 25 documents performance characteristics, benchmarking infrastructure, and scalability. Section 26 lists known limitations honestly. Section 27 outlines future work.

Readers wanting to get started quickly should read sections 2 and 12, then refer to individual sections as needed.

2 Data Model

This section defines the core abstractions that all other components build upon.

2.1 Devices

`netsim` models five device types, all implementing the Device trait:

Table 1. Device types and their roles.

Type	Rust type	Description
Router	Router	Full L3 device with RIB/FIB, protocol daemons (OSPF v2/v3, BGP, IS-IS, RIP, MPLS, RSVP-TE, BFD, LLDP, LACP, VRRP, DHCP Relay, IGMP, PIM-SM), bridge domains (STP), MFIB, VRFs
Host	Host	L3 endpoint with a default gateway; generates and sinks traffic; IGMPv2 group membership
Switch	Switch	L2 device with MAC learning and VLAN support
Hub	Hub	L1 device that floods all frames to all ports
External	TapDevice	Hybrid bridge that binds a simulated interface directly to a Linux/macOS host TAP interface for real-world traffic injection
Wire	Wire	Point-to-point link with latency, loss, jitter, and capacity modelling

Note

A sixth device type, `DnsServer`, implements the `Device` trait for internal DNS resolution (used by the `dig` command). It is not stored in the engine’s type-segregated vectors and cannot be instantiated from YAML—it exists solely to provide DNS query/response handling within the simulation.

Design Decision

Why five types instead of a generic “Node”?

Type-segregated storage (`Vec<Option<Box<Router>>>`, `Vec<Option<Box<Host>>>`, etc.) enables Rayon’s `par_iter_mut` to process each device type independently without unsafe code [2]. A single `Vec<Box<dyn Device>>` would require unsafe downcasting or runtime locks for parallel mutation. The five-type design trades some code duplication for zero-cost safe parallelism.

Each device is identified by a `DeviceId(u32)` and contains a vector of interfaces. Interfaces are identified by `InterfaceId(u32)` and carry IPv4/IPv6 addresses, MAC addresses, and link-state information.

2.2 Wires and Interface QoS

A `Wire` connects exactly two interfaces on different devices. It models latency (frames spend a configurable number of ticks in transit via a `VecDeque` of in-flight frames), probabilistic packet loss using a seeded PRNG, and jitter as random variation on the base latency.

Interfaces can be configured with `strict_rate_limit_bps` settings. When simulated traffic generators exceed this capacity, the interface’s transmit queues fill and begin tail-dropping frames, accurately simulating network congestion and allowing validation of QoS and traffic engineering failover logic.

All internal identifiers (`DeviceId`, `InterfaceId`, `WireId`) are `u32` indices into their respective storage vectors, providing $O(1)$ lookup. `RouterId` is an `Ipv4Addr` used as the OSPF/BGP router identifier.

3 Forwarding Architecture

3.1 RIB/FIB Separation

Each router maintains two distinct data structures. The **RIB** (Routing Information Base) stores every route learned from every protocol, tagged with source protocol, administrative distance, and metric. The **FIB** (Forwarding Information Base) retains only the best routes selected from the RIB and is the sole structure consulted by the forwarding engine when processing a packet.

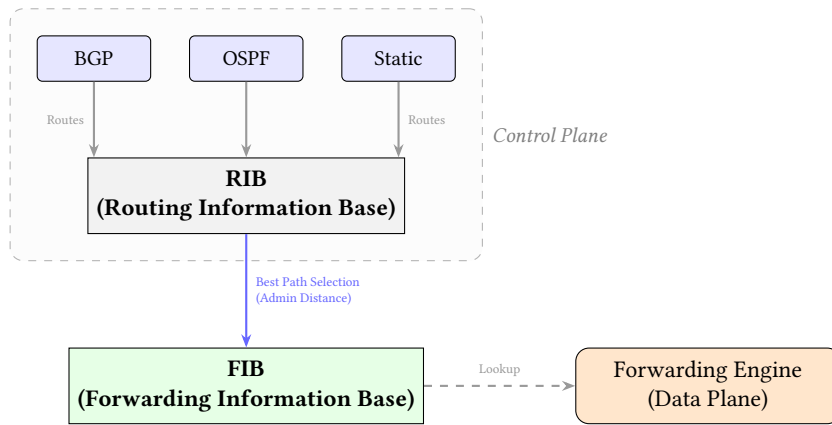


Figure 2. RIB/FIB Separation. Multiple routing protocols populate the global RIB. The router executes best-path selection based on Administrative Distance to program the active FIB used by the forwarding engine.

Design Decision

Why separate RIB and FIB?

Real routers maintain this separation because the control plane (protocol daemons) and data plane (forwarding ASIC) operate independently. netsim mirrors this to ensure that protocol interactions (e.g., OSPF and BGP both advertising the same prefix) are resolved correctly via administrative distance, and to enable accurate modelling of RIB→FIB installation delays.

3.2 Dual-Index FIB

The FIB uses two co-maintained data structures:

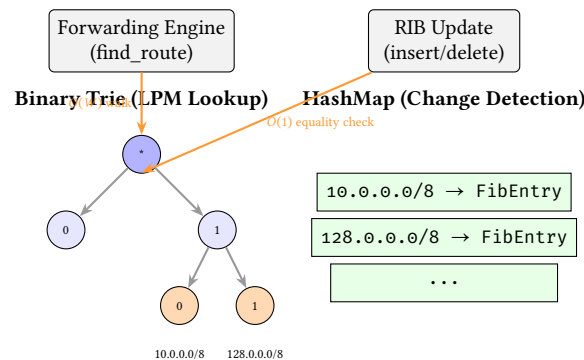


Figure 3. Dual-Index FIB Architecture. The Forwarding Engine uses the Binary Trie for fast Longest-Prefix-Match lookups, while RIB updates use the HashMap for $O(1)$ exact-match change detection to avoid spurious convergence resets.

1. A **binary trie** for longest-prefix-match (LPM) lookups. Implemented as `Vec<TrieNode>` where each `TrieNode` has `[Option<usize>; 2]` children and an optional `FibEntry`. LPM walks the trie bit by bit from MSB to LSB, tracking the last seen entry. Worst case: 32 steps for IPv4.
2. A **HashMap** (`HashMap<Ipv4Net, FibEntry>`) for $O(1)$ exact-match lookups during FIB update change detection.

To efficiently handle high-churn routing environments (e.g., Internet-scale BGP), netsim implements **Incremental FIB Updates**. Adding or removing a prefix from the RIB results in an $O(\log N)$ update to the Trie rather than an $O(N)$ full FIB rebuild. Atomic multi-prefix updates allow hundreds of route changes to be committed to the FIB in a single batch, drastically improving simulator convergence times on large topologies.

Design Decision

Problem: FIB updates must not trigger spurious convergence resets. If a protocol daemon re-announces an unchanged route, and the FIB's change flag is set, convergence detection restarts unnecessarily.

Options considered: (1) Trie-only FIB with entry comparison on every update ($O(W)$ per lookup + $O(W)$ per update comparison). (2) HashMap-only FIB ($O(1)$ updates but $O(n)$ LPM via linear scan). (3) Dual-index with trie for LPM and HashMap for change detection.

Decision: Dual-index. The HashMap enables $O(1)$ equality checking during updates (`old_entry == new_entry?`), and the trie provides $O(W)$ LPM for forwarding. Both are updated atomically via `replace_entries`.

Trade-off: Double memory for the FIB. In practice, FIB memory is negligible compared to protocol state (BGP RIB, OSPF LSDB). Benchmarks demonstrate that the $O(1)$ equality check prevents millions of redundant $O(W)$ LPM tree traversals per second, significantly reducing the convergence tail in high-churn scenarios.

3.3 ECMP

When the RIB installs multiple equal-cost routes to the same prefix, the FIB stores all next-hops. At forwarding time, a flow-consistent hash selects among them:

$$\text{hash}(\text{src}, \text{dst}, N) = (\text{src} + (\text{dst} \lll 16)) \pmod{N} \quad (1)$$

where $\lll$ denotes a bitwise left rotation.

This ensures all packets of a given (src, dst) flow traverse the same path, while distributing distinct flows across available next-hops.

Warning

The ECMP hash does not include L4 ports. This means all traffic between a given (src_ip, dst_ip) pair follows the same path regardless of application. This is simpler than real router implementations (which typically hash on the 5-tuple) but sufficient for routing-level validation.

4 Control Plane Routing

netsim models the traditional routing protocol stack where Interior Gateway Protocols (IGPs) discover the topology and build the underlay, while BGP handles policy, exterior routing, and overlays. Each protocol runs its own state machine and populates the global RIB, which arbitrates conflicts using administrative distance. This modular design accurately reproduces the complex interactions and race conditions that occur in real-world networks during convergence.

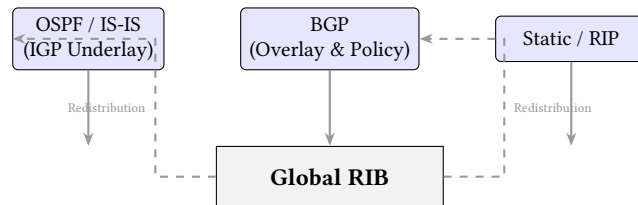


Figure 4. Control Plane Protocol Interactions. Protocols independently compute routes and submit them to the Global RIB, which arbitrates conflicts using administrative distance. The RIB can also redistribute routes back into protocols.

netsim implements the following routing and infrastructure protocols. Each implementation targets RFC-correct behaviour at tick granularity, not vendor-specific quirks.

Table 2. Protocol coverage.

Protocol	RFCs	Scope
OSPF v2	2328 [3]	Areas, stub/NSSA, virtual links, SPF with incremental recalculation
IS-IS	ISO 10589 [4, 5]	L1/L2, wide metrics, TLV extensibility
BGP-4	4271 [6]	iBGP, eBGP, route reflection, confederations, communities, AS-path filtering
BGP EVPN	7432 [7]	Type-2/3/5, RT import/export, ARP suppression
MPLS/LDP	5036 [8]	Label distribution, PHP, ECMP label stacking
RSVP-TE	3209 [9]	Explicit paths, bandwidth reservation, FRR
BFD	5880 [10]	Async mode, microsecond-granularity timers (mapped to ticks)
LACP	802.1AX [11]	Fast/Slow timers (100/3000ms), LAG bundling, actor/partner state machines
LLDP	802.1AB [12]	Periodic hellos, neighbour table, TLV parsing
VXLAN	7348 [13]	Encap/decap, head-end replication
RIPv2	2453 [14]	Periodic/triggered updates, split horizon, 15-hop metric limit, route poisoning
RIPng	2080 [15]	IPv6 distance-vector routing
STP	802.1D [16]	Root bridge election, port roles (Root/Designated/Blocking), topology change detection
VRRP v2/v3	3768 [17], 5798 [18]	Master/Backup election, virtual IP, tracked interfaces, priority decrements, virtual MAC (00:00:5E:00:01:VRID)
DHCP Relay	2131 [19]	Broadcast-to-unicast conversion, multiple helper addresses, GIADDR population, Option 82 (Circuit ID, Remote ID)
IGMP v2/v3	2236 [20], 3376 [21]	Membership Reports/Leaves, Query, querier election, Source Filtering (INCLUDE/EXCLUDE)
PIM-SM	7761 [22]	Hello/neighbor discovery, DR election, RPF resolution via static RP, MFIB population
OSPFv3	5340 [23]	IPv6 OSPF with link-local addressing
SRv6	8754 [24], 8986	SRH codec, SID table, End/End.X behaviours, headend encap (v2.2, in progress)

Note

All protocol timers (OSPF dead interval, BGP hold time, BFD detection multiplier, LACP fast/slow, RIP update interval, VRRP advertisement interval, PIM hello interval, IGMP query interval) are converted to integer tick counts at build time. A 10-second OSPF dead interval at 1 ms/tick resolution becomes 10,000 ticks.

4.1 OSPF

The OSPF implementation follows RFC 2328 [3] with a complete neighbour state machine progressing from Down through Init, 2-Way, ExStart, Exchange, and Loading to Full. Each router maintains an independent LSDB; SPF is triggered only on LSDB changes rather than running every tick, matching production router behaviour where idle periods produce no CPU load. Area support includes backbone (Area 0), stub, NSSA, and virtual links, enabling realistic multi-area designs.

4.2 BGP

BGP (RFC 4271 [6]) follows a full FSM from Idle through Established, with both iBGP and eBGP session types. The best-path selection implements a 13-step decision process matching Cisco IOS ordering (local-pref, AS-path length, origin type, MED, eBGP over iBGP, IGP cost, router-ID tiebreak). Route reflection with cluster-ID and

originator-ID enables scalable iBGP meshes. Three address families are supported: IPv4 unicast, VPNv4 (for L3VPN), and EVPN (for VXLAN fabrics).

The interaction between OSPF and BGP is where netsim's tick-based model proves its value: BGP next-hop resolution depends on the IGP FIB, so an OSPF reconvergence that changes the shortest path to a BGP next-hop will trigger BGP best-path recalculation—exactly as it would on real hardware. This cross-protocol coupling is invisible in static analysis tools but critical for validating failover scenarios.

4.3 RIP

netsim implements both RIPv2 (RFC 2453 [14], IPv4) and RIPng (RFC 2080 [15], IPv6). The hop-count metric enforces a 15-hop limit (16 = infinity), with split horizon and poisoned reverse to prevent count-to-infinity. Updates are sent at a default 30-second interval, with triggered updates on topology changes for faster convergence. RIP is enabled per-interface via `rip: true` in the YAML topology.

Design Decision

Why include RIP in a modern simulator?

RIP is rarely deployed in new networks, but it remains valuable for three reasons: (1) it is the simplest routing protocol to understand, making netsim useful for educational settings; (2) legacy networks still run RIP in stub areas; (3) RIP's distance-vector behaviour (count-to-infinity, split horizon) exercises different convergence dynamics than OSPF's link-state SPF, broadening the convergence predicate's test coverage.

5 Data Plane & Encapsulation

5.1 Hybrid Real-Sim Data Plane (TUN/TAP Bridge)

While netsim natively generates synthetic traffic for assertions, it also supports bridging the simulated data plane to the physical host via Linux/macOS TAP interfaces. An `External` device type binds non-blockingly to a host OS TAP file descriptor (e.g., `/dev/net/tun`).

During the engine tick loop, raw Ethernet frames received from the physical OS are ingested into the simulation, routed through the virtual FIBs/LFIBs, and egressed back out to the TAP interface. This enables true hybrid testing: operators can use real network diagnostic tools (like `nmap`, `ping`, or `iperf`) from their laptop directly into the simulated BGP core, or run a real web server that depends on the simulated network for reachability.

5.2 MPLS and RSVP-TE

netsim implements the full MPLS label stack (RFC 3031 [25]) with push, swap, and pop operations. LDP (RFC 5036 [8]) automatically distributes labels following the IGP topology, while RSVP-TE (RFC 3209 [9]) provides explicit label-switched paths with bandwidth constraints for traffic engineering. Penultimate hop popping (PHP) is supported, removing the outermost label one hop before the egress router to reduce lookup overhead. L3VPN is built on top, with per-VRF import/export using VPNv4 route targets to isolate customer routing domains.

5.3 SRv6 (Segment Routing over IPv6)

netsim implements the SRv6 data plane per RFC 8754 [24] (Segment Routing Header) and RFC 8986 [26] (SRv6 Network Programming). This is an active development area (milestone v2.2).

5.3.1 Segment Routing Header (SRH) & Packet Walk

The SRv6 packet walk intercepts the traditional IPv6 lookup by interposing a SID table. When a packet arrives, the forwarding engine inspects the IPv6 Destination Address (DA). If the DA matches a local SID in the SID table, the associated endpoint behaviour is executed; otherwise, normal IPv6 FIB forwarding resumes.

For an **End** behaviour (transit), the router examines the Segment Routing Header (SRH). It decrements the `segments_left` field, copies the next segment from the SRH into the IPv6 DA, and loops the packet back to the start of the IPv6 forwarding pipeline. For an **End.X** behaviour, the packet is cross-connected out a specific layer-3 interface, bypassing the FIB lookup entirely.

Headend encapsulation statically wraps matching transit traffic. A policy matches an incoming prefix, wraps the inner packet in a new outer IPv6 header and an SRH containing the segment list, and sets the outer DA to the first active segment.

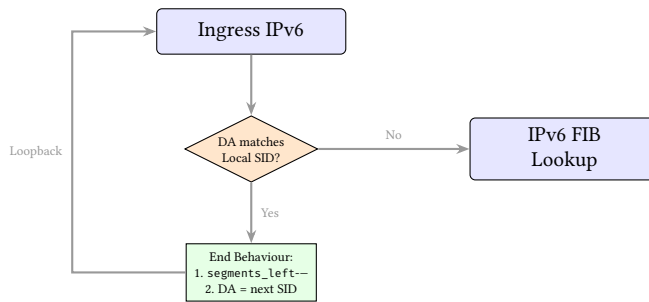


Figure 5. SRv6 Packet Walk. The forwarding engine intercepts the IPv6 Destination Address. A local SID match triggers SRv6 endpoint processing, rewriting the DA and looping the packet back for forwarding.

5.3.2 YAML Configuration

SRv6 is configured per-device in the topology file:

```

devices:
- name: R1
  router_id: 1.1.1.1
  srv6:
    locator: "2001:db8:1::/48"
    sids:
      - sid: "2001:db8:1:100::"
        behavior: end.x
        interface: eth0
    encap:
      - dst: "10.0.3.0/24"
        segments:
          - "2001:db8:2::1"
          - "2001:db8:3::1"
  
```

Note

Transit forwarding is SRH-blind: routers that are not the target of a SID forward on the outer IPv6 DA only. SRH processing occurs only when the DA matches a local SID table entry. This matches real router behaviour and means that SRv6 does not require any changes to existing IPv6 FIB lookup logic.

Warning

SRv6 is under active development (milestone v2.2, Phase 125–126). The SRH codec and SID table are complete. Endpoint behaviour wiring into the forwarding pipeline, `show srv6 sid` commands, and service SID types (End.DT4, End.DT6) are planned for upcoming phases. Flavours (PSP, USP, USD) and SRv6 Policy with colour/endpoint steering are scoped for Phase 126.

5.4 VXLAN

VXLAN provides L2 overlay encapsulation. The data plane handles UDP/IP encapsulation with a VNI (VXLAN Network Identifier) and split-horizon filtering to prevent BUM (Broadcast, Unknown Unicast, Multicast) forwarding loops. For a detailed discussion of the VXLAN control plane (BGP EVPN) and bridge domain integration, see Section 8.

6 Infrastructure & Edge

netsim models layer 2 and edge infrastructure protocols to accurately reproduce failure detection, link aggregation, and host connectivity. Figure 6 shows how these protocols layer in a typical campus deployment: LLDP discovers physical neighbours, STP prevents L2 loops, LACP bonds links, BFD detects failures, and VRRP/DHCP Relay operate at the edge to provide gateway redundancy and address assignment.

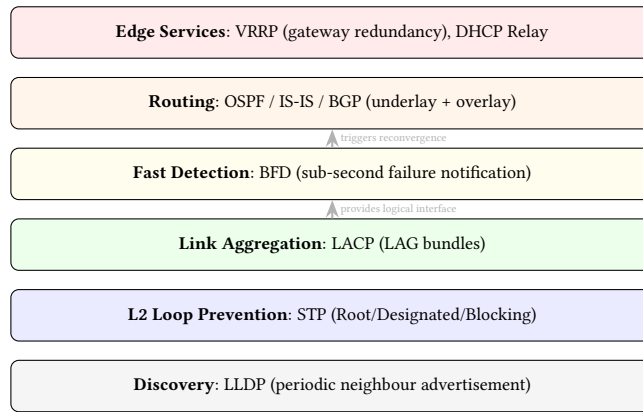


Figure 6. Campus protocol stack. Each layer operates at a different level of the network: LLDP discovers topology, STP prevents loops, LACP aggregates links, BFD detects failures and notifies routing protocols, and VRRP/DHCP Relay serve the network edge.

6.1 BFD, LACP, LLDP

In a production campus or data centre, three infrastructure protocols operate below the routing layer. **BFD** (Bidirectional Forwarding Detection) [10] provides sub-second failure detection: when a BFD session drops, it triggers immediate IGP/BGP reconvergence rather than waiting for hold-timer expiry—often reducing failover from 40 seconds to under 1 second. **LACP** [11] bundles physical links into a single logical interface; netsim models both fast (100/300 tick) and slow (3000/9000 tick) timer modes. **LLDP** [12] handles periodic neighbour discovery and is deliberately excluded from quiescence checks because it never stops sending.

6.2 STP

Spanning Tree Protocol [16] prevents L2 forwarding loops on Switch devices through root bridge election (lowest bridge ID wins), port role assignment (Root, Designated, Blocking), and topology change notification. When a link goes down, TCN BPDUs propagate across the switched domain, triggering MAC table flushes to prevent stale forwarding.

6.3 VRRP

VRRP (RFC 3768/5798 [17, 18]) provides first-hop gateway redundancy by floating a virtual IP address between a Master and one or more Backup routers. Priority-based election determines the Master; tracked interfaces dynamically adjust priority so that an upstream link failure triggers failover. The standard virtual MAC (00:00:5E:00:01:VRID) avoids gratuitous ARP storms. Both IPv4 (VRRPv2) and IPv6 (VRRPv3) are supported.

6.4 DHCP Relay

The DHCP Relay Agent (RFC 2131 [19]) converts broadcast Discover/Request packets into unicast messages forwarded to configured helper addresses, populating the GIADDR field so the server selects the correct subnet pool. Option 82 (Circuit ID, Remote ID) is appended for downstream accounting; pre-existing Option 82 from untrusted sources is dropped. netsim models the relay path rather than server address-pool logic, which is sufficient to validate enterprise campus designs where relay traversal through firewalls and VRFs is the common failure mode.

6.5 Multicast: IGMP, PIM-SM, and MFIB

netsim implements a multicast forwarding foundation comprising three cooperating layers. At the edge, **IGMPv2** [20] and **IGMPv3** [21] manage group membership: hosts send Membership Reports to join multicast groups, routers track per-interface group state (including IGMPv3 Source Filtering for INCLUDE/EXCLUDE modes), elect a querier (lowest IP), and age out stale memberships. General Queries are sent periodically by the elected querier.

In the core, **PIM-SM** [22] discovers neighbors via Hello packets and elects a Designated Router (DR) per segment. The PIM instance populates MFIB RPF state for $(*, G)$ entries by resolving the Rendezvous Point (RP) through static RP configuration and performing a unicast FIB lookup toward the RP to determine the incoming interface and RPF neighbor.

The **MFIB** ties these together with a dedicated multicast FIB storing $(*, G)$ and (S, G) entries with IOS-style flags (Wildcard, Connected, SPT, Register-Suppressed). Each entry tracks incoming interface, RPF neighbor, and an outgoing interface list (OIL). IGMP populates OIL entries from downstream group membership, while PIM resolves RPF state from the unicast routing table. Static RP mappings (group prefix $\rightarrow$ RP address) are configured via YAML, with longest-prefix matching to resolve the RP for each group.

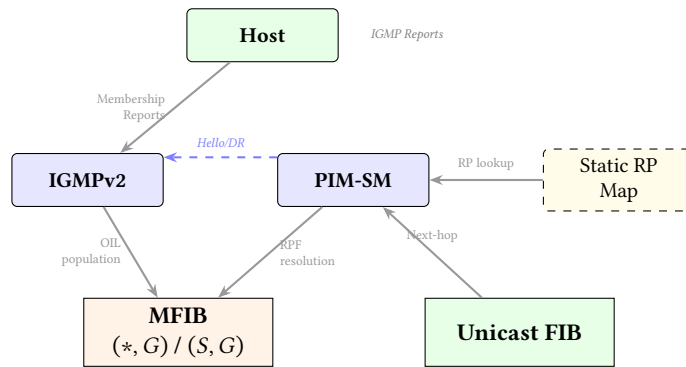


Figure 7. Multicast control plane data flow. IGMP learns group membership from hosts and populates the MFIB outgoing interface list. PIM-SM resolves RPF state by looking up the RP in the static RP map and performing a unicast FIB lookup toward the RP.

Multicast is configured per-device in the YAML topology:

```

devices:
- name: rp-router
  type: router
  multicast:
    rp:
      239.0.0.0/8: 10.0.0.1 # Static RP for all 239.x groups

- name: receiver
  type: host
  multicast_groups: # Host joins these groups via IGMP
  - 239.1.1.1
  igmp_report_interval_ticks: 5000

```

Design Decision

Multicast scope and design rationale.

The current implementation focuses on the multicast control plane foundation: group membership tracking (IGMP), neighbor discovery and DR election (PIM-SM), and forwarding state management (MFIB with RPF). This is sufficient to validate RP reachability, RPF correctness, and OIL population in multi-hop topologies. Full PIM Register/Join/Prune messaging, SPT switchover, and MLD (IPv6) are planned as future extensions.

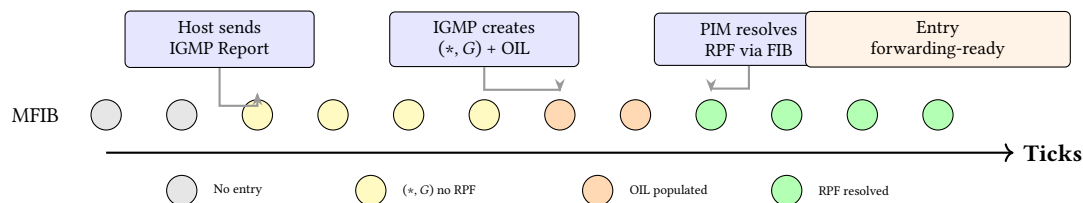


Figure 8. MFIB entry lifecycle. A host's IGMP Membership Report triggers $(*, G)$ creation. IGMP populates the outgoing interface list (OIL) from downstream receivers. PIM resolves the RPF neighbor and incoming interface by looking up the RP in the unicast FIB. The entry is forwarding-ready once both OIL and RPF state are populated.

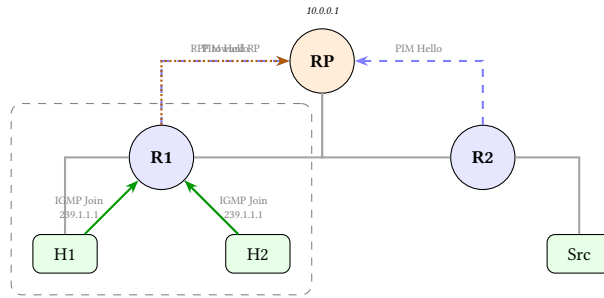


Figure 9. Example multicast topology. Hosts H1 and H2 send IGMP Membership Reports for group 239.1.1.1 to router R1. R1's PIM instance discovers the RP (10.0.0.1) via static configuration and resolves the RPF path through the unicast (OSPF) FIB. PIM Hello packets maintain neighbor adjacencies between all routers.

7 Declarative Topology & Chaos Engineering

7.1 YAML Topology Format

A netsim topology is a single YAML file whose top-level sections fall into four layers, shown in fig. 10.

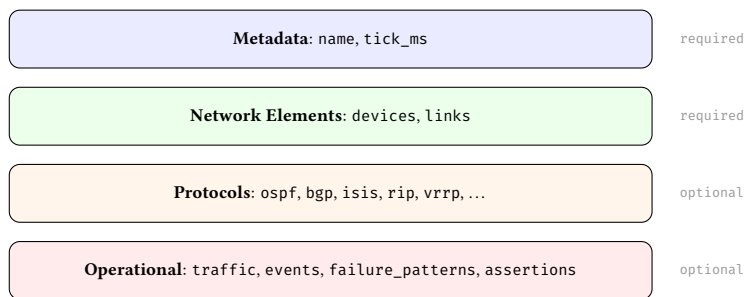


Figure 10. Topology file structure. A single YAML file is organised into four layers: metadata, network elements (devices and links), protocol configuration, and operational concerns (traffic, chaos, assertions).

7.2 Pattern-Based Selectors

Rather than configuring each device individually, netsim supports selectors that match groups of devices or links:

Listing 1. Pattern-based selectors.

```
1 impairment_profiles:
2   - name: wan-latency
3     selector: { by_tier: [core, edge] }
4     latency_ms: 10
5     loss_percent: 0.01
6   - name: access-links
7     selector: { name_pattern: "access-*" }
8     latency_ms: 2
```

Selectors match by tier, name glob, or geographic distance, reducing a 486-device topology from 630+ per-link configs to a handful of profiles.

Design Decision

Why selectors instead of per-device configuration?

A 486-device topology would require 630+ link configurations without selectors. Pattern-based selectors reduce this to a handful of profiles, making topologies readable and maintainable. The trade-off is that selectors can match unintended devices if patterns are too broad—netsim logs which devices each selector matched during the build stage to aid debugging.

7.3 Declarative Assertions

To enable automated CI/CD validation, netsim supports declarative assertions that are evaluated after convergence. This allows operators to encode their network intent directly in the topology file:

Listing 2. Declarative assertions.

```
1 assertions:
```

```

2 # End-to-end reachability intent
3 - type: reachability
4   source: host-db
5   destination: 10.0.1.1
6   expected: true
7
8 # Protocol state invariant
9 - type: ospf_neighbor_state
10  router: core-spine1
11  interface: eth0
12  expected_state: Full

```

If any assertion fails, netsim exits with a non-zero status code and provides a detailed diagnostic report, making it ideal for integration into automated infrastructure pipelines.

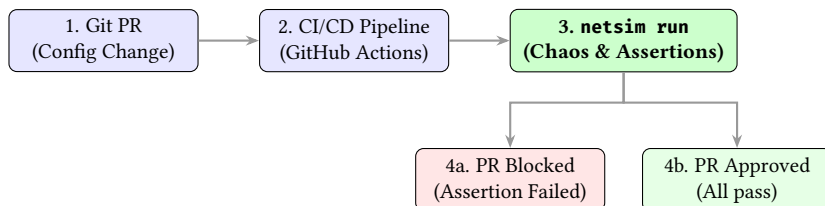


Figure 11. CI/CD Pipeline Integration. Declarative assertions allow netsim to automatically block PRs that would introduce routing loops or isolation, preventing operator-induced outages before deployment.

7.3.1 Assertion Taxonomy

The assertion framework provides 19 built-in assertion types organised into five categories. Each assertion is declared in the YAML topology via a discriminated type: tag and evaluated after convergence.

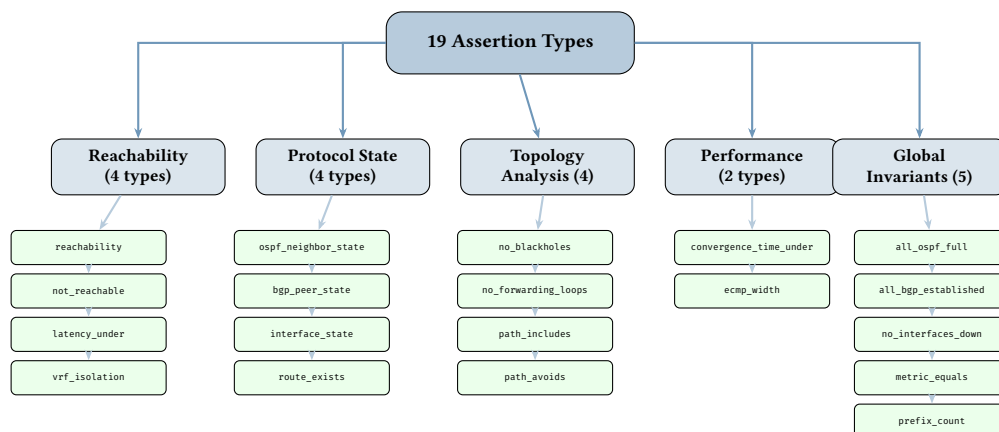


Figure 12. Assertion Taxonomy. The 19 built-in assertion types are organised into five categories: reachability probes, protocol state checks, topology analysis (FIB walks), performance budgets, and network-wide invariants. All are expressed declaratively in YAML and evaluated after convergence.

The assertion taxonomy in fig. 12 shows all 19 built-in types; the full parameter reference is in table 18 (section A). In practice, most topologies use a handful of assertions spanning two or three categories.

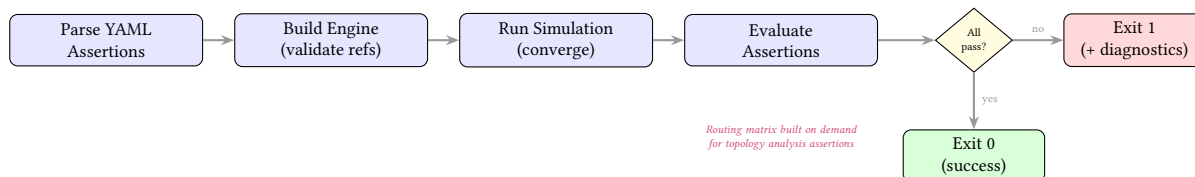


Figure 13. Assertion Evaluation Pipeline. Assertions are parsed from YAML, validated during engine build, and evaluated after convergence. Topology analysis assertions (blackhole/loop detection) lazily construct the routing matrix only when needed.

7.4 Failure Patterns

netsim supports four failure pattern types for chaos engineering [27]:

Table 3. Failure pattern types.

Type	Behaviour
probabilistic	At each <code>check_interval</code> , fail with probability p . Recovery after n ticks.
periodic	Deterministic on/off cycling with configurable up/down durations.
flapping	Rapid state oscillation (up→down→up) to stress convergence.
cascade	Dependency-triggered chain failures: when a primary link fails, dependent links/devices also fail.

Each `FailurePattern` is expanded into a sequence of `ScheduledEvents` at build time. This cleanly separates *what should happen* (the pattern) from *when it happens* (the schedule), and ensures deterministic replay.

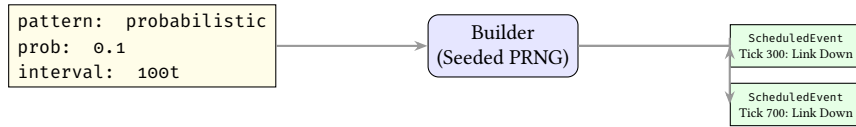


Figure 14. The Chaos Event Scheduler. Probabilistic YAML definitions are deterministically expanded into concrete scheduled events at build time using a seeded PRNG.

Additionally, geographic latency can be formulated deterministically based on physical distance. For links configured with `by_distance: true`, the latency in ticks L is calculated as:

$$L = \left\lceil \frac{d(coord_1, coord_2)}{c_{\text{fiber}}} \times \frac{1}{\text{tick_ms}} \right\rceil \quad (2)$$

where d is the Haversine distance, and c_{fiber} is the speed of light in fiber optic cable ($\approx 2 \times 10^8$ m/s).

8 EVPN/VXLAN at Simulation Abstraction

Data centre fabric validation is a primary use case for `netsim`. This section documents the EVPN/VXLAN [7, 28] implementation in detail.

8.1 Problem

Validating a leaf-spine EVPN/VXLAN fabric typically requires `Containerlab` with FRR or EOS containers. This consumes gigabytes of RAM (each FRR container runs a full Linux userspace), takes minutes to boot, and produces non-deterministic convergence behaviour. An operator wanting to validate “does my EVPN fabric converge correctly after a spine failure?” should not need to wait 3 minutes and hope for consistent results.

8.2 Unified Router as L2+L3 Device

Unlike many simulators that model switches and routers as separate device types, `netsim`’s `Router` serves as a unified L2+L3 device. Each `Router` can hold zero or more *bridge domains*, making it capable of acting as a pure L3 router, a VTEP with bridging, or an IRB (Integrated Routing and Bridging) gateway—all within the same device abstraction.

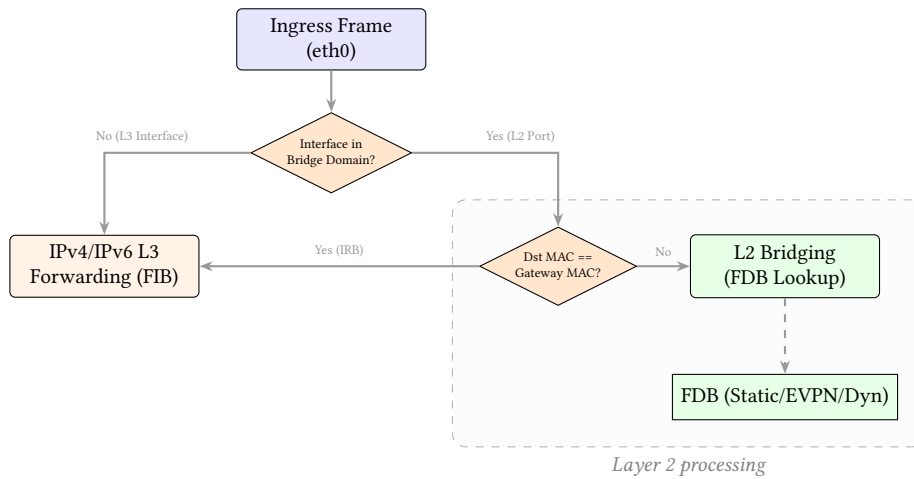


Figure 15. Unified L2/L3 Router Abstraction. Incoming frames branch based on interface bridge domain membership. IRB (Integrated Routing and Bridging) allows L2 frames addressed to the gateway MAC to directly bridge into the L3 forwarding path.

Each router maintains a map of bridge domains keyed by VNI, and an interface-to-VNI lookup table that drives the L2/L3 forwarding split. If an ingress interface is mapped to a bridge domain, the frame enters `process_frame_l2`. Otherwise, it proceeds to IPv4/IPv6 L3 forwarding. Within the L2 path, an IRB check fires first: if the frame’s destination MAC matches the bridge domain’s gateway MAC, the frame is routed into the associated VRF, bypassing L2 bridging entirely.

8.3 Bridge Domain and FDB

Each bridge domain maintains an independent forwarding database, illustrated in fig. 16.

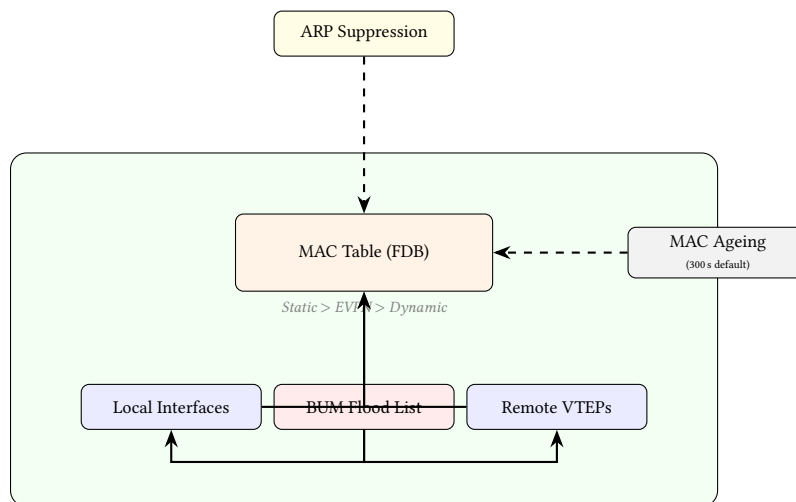


Figure 16. Bridge domain internals. Each domain maintains an independent MAC table with a three-tier entry hierarchy (Static > EVPN > Dynamic), a BUM flood list spanning local interfaces and remote VTEPs, ARP suppression to reduce broadcast traffic, and timer-based MAC ageing for dynamically learned entries.

FDB entries have three types with a strict override hierarchy:

1. **Static:** Manually configured, never ages, highest priority. Cannot be overridden by dynamic learning.
2. **EVPN:** Control-plane learned via BGP Type-2 routes. Never ages (removed only on BGP withdrawal). Carries MAC mobility sequence numbers for move detection.
3. **Dynamic:** Data-plane learned on local ingress frames. Ages out after `mac_timeout` ticks. Cannot overwrite static or EVPN entries.

BTreeMap is used instead of HashMap throughout for deterministic iteration order—critical for reproducible BUM flooding and consistent `show` command output.

Note

MAC ageing events deliberately do *not* set `fdb_changed_this_tick`. Ageing is routine housekeeping, not a convergence-relevant state change. Only new learning events and control-plane updates affect the convergence predicate.

8.4 BGP EVPN Control Plane

netsim implements all five EVPN route types defined in RFC 7432 [7] and RFC 9136:

Table 4. EVPN NLRI types.

Type	Name	Purpose
1	EAD (Ethernet Auto-Discovery)	Mass withdrawal on ES failure; aliasing for multi-homed load balancing. Two subtypes: per-ES (<code>ethernet_tag=0</code>) and per-EVI (<code>ethernet_tag=VNI</code>).
2	MAC/IP Advertisement	Advertises host MAC and optional IP binding. Used for remote MAC learning and ARP suppression.
3	Inclusive Multicast Ethernet Tag (IMET)	Establishes the VTEP flood list for BUM traffic replication per VNI.
4	Ethernet Segment Route	PE discovery for multi-homing. Used to build the candidate list for DF election.
5	IP Prefix Route	Advertises IP prefixes for inter-subnet routing via the VXLAN fabric (symmetric IRB).

8.5 VXLAN Encapsulation and Decapsulation

The VXLAN data plane handles encapsulation and decapsulation in the Router's forwarding pipeline:

Encapsulation path. When a bridge domain needs to forward a frame to a remote VTEP:

1. **MTU check:** Verify that the inner frame plus 42 bytes of VXLAN/UDP/IP overhead fits within the egress interface MTU (default 1500). Drop with counter if exceeded.
2. **UDP source port:** Computed from a hash of the inner frame's source and destination MACs (mapped to range 49152–65535). This provides ECMP entropy for the underlay, per RFC 7348.
3. **Inner frame encoding:** A compact binary format with marker bytes (NSFR), version, MAC addresses, EtherType, TTL, and optional L3/L4 metadata.
4. **Multi-homing metadata:** When the frame originates from a multi-homed Ethernet Segment, a 15-byte metadata block (NSMH marker + 10-byte ESI) is appended for split-horizon filtering at the remote VTEP.
5. **Underlay forwarding:** The outer IP packet is forwarded via the underlay FIB using the destination VTEP IP.

Decapsulation path. On receiving a VXLAN packet (UDP dst port 4789):

1. Parse VXLAN header, extract VNI.
2. Look up the bridge domain by VNI.
3. Detect and strip optional multi-homing metadata.
4. Decode the inner frame.
5. Call `process_frame_l2` with `from_vxlan=true` and the ingress ESI (if present).

The `from_vxlan=true` flag is the critical hairpin prevention mechanism: it prevents re-replication of decapsulated frames back to remote VTEPs, which would cause infinite flooding loops.

8.6 MAC Mobility

MAC mobility detection follows RFC 7432 Section 15.2. Each EVPN Type-2 route carries a MAC Mobility extended community with a sequence counter and a sticky flag. When a MAC is learned on a new interface, the local VTEP increments the sequence number and re-originates the Type-2 route. On receiving a remote route, the selection rule is: higher sequence wins; on tie, lower originating IP wins for stability. MAC sequences are tracked per (VNI, MAC) pair inside the BGP instance.

8.7 ARP Suppression

When a host sends an ARP request, the local VTEP checks its EVPN Type-2 routes for a matching IP→MAC binding. If found, the VTEP replies directly without flooding the ARP request across the VXLAN fabric. This reduces BUM traffic significantly in large fabrics. ARP suppression is enabled by default in BridgeDomainConfig and can be disabled per bridge domain.

8.8 Snapshot-Then-Apply Synchronisation

Design Decision

Problem: How should EVPN Loc-RIBs be synchronised across routers in a tick-based simulation?

Options considered:

1. **Per-message queuing:** Model individual BGP UPDATE messages with per-session send/receive queues. High fidelity but complex ($O(\text{sessions} \times \text{routes})$ queue operations per tick) and introduces ordering-dependent non-determinism.
2. **Snapshot-then-apply:** At each tick, snapshot all EVPN Loc-RIBs, then have each router apply remote updates from the snapshot.

Decision: Snapshot-then-apply (Phase P2.5 in the tick pipeline).

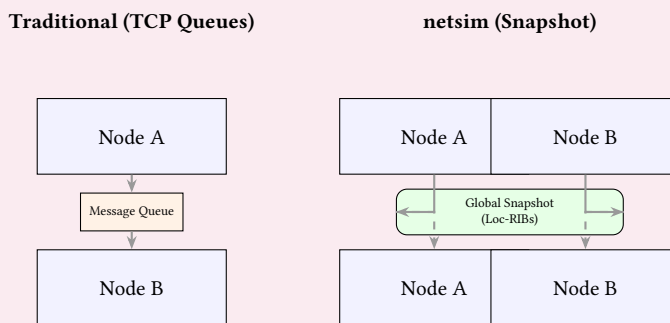


Figure 17. EVPN Snapshot Abstraction. Instead of modeling asynchronous TCP queues and individualized BGP update messages (left), netsim captures a global snapshot of all EVPN Loc-RIBs at phase P2.5 and atomically applies changes to all peers (right), guaranteeing deterministic state replication.

This preserves `peer_router_id` metadata to prevent route-reflector-induced flapping while avoiding per-message queue complexity.

Trade-off: Cannot model BGP UPDATE message ordering bugs or per-session backpressure. This is an acceptable trade-off for a tool that targets configuration validation, not BGP implementation testing.

The synchronisation pass performs split-horizon filtering: routes are not advertised back to the peer from which they were learned, checked by comparing `route.peer_router_id` against the destination peer's router ID. Importantly, `peer_router_id` is preserved from the original route—not replaced with the reflecting router's ID—to prevent EVPN flap in route-reflector topologies.

8.9 EVPN Multi-Homing

netsim supports EVPN all-active multi-homing for redundant host connectivity, implementing the full Type-1/Type-4 route exchange.

An **Ethernet Segment Identifier (ESI)** groups interfaces on different VTEPs connecting to the same multi-homed host or switch. ESI labels are derived deterministically via FNV-1a hash, clamped to the 20-bit MPLS label range. **Designated Forwarder (DF) election** uses RFC 7432 service-carving with modulo-based selection (`vni % len(candidates)`), where candidates are discovered via Type-4 Ethernet Segment routes and sorted by VTEP IP for determinism.

Loop prevention operates at two levels. At BUM flood time, a VXLAN-decapped frame is only forwarded to a local interface if that interface's VTEP is the DF for the given VNI. At unicast time, if the ingress ESI matches the egress interface's ESI, the frame is dropped to prevent hairpin loops.

Aliasing ensures traffic is correctly distributed across multi-homing peers: when a received Type-2 route carries a non-zero ESI matching a local Ethernet Segment, the FDB entry is installed as local rather than remote. When the ESI is known but the local interface is absent, deterministic load balancing distributes traffic across all known PEs for that ESI using a hash of (VNI, MAC).

9 CLI Tool & Interactive Daemon

The `netsim` CLI is designed around three principles: *operator familiarity* (IOS-style commands that network engineers already know), a *vendor-neutral canonical engine* (one protocol implementation, multiple output skins), and *zero-configuration discovery* (run `netsim` with no arguments to browse live daemons and attach to any device). It supports both batch execution (run a topology, collect results, exit with a pass/fail code) and interactive operation (attach to a live simulation, inspect state, inject failures in real time).

9.1 CLI Subcommands

The CLI is the primary interface to `netsim`, organised into four command families shown in fig. 18.

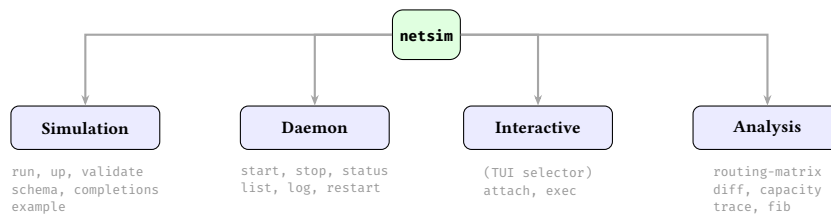


Figure 18. CLI command taxonomy. `run` executes a simulation to completion; `up` combines daemon start and interactive attach; the analysis commands operate on live daemons or saved state.

9.2 Operational Emulation via Vendor Skinning

While maintaining a single unified lock-free protocol engine, `netsim` overlays a zero-overhead vendor-specific CLI translation layer supporting Cisco (IOS/NX-OS), Juniper (JunOS), Arista (EOS), Nokia (SR OS), and Huawei (VRP).

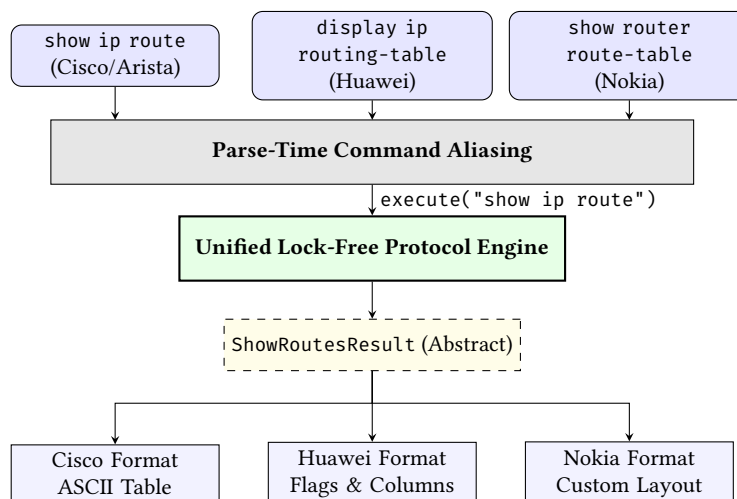


Figure 19. The Zero-Overhead Vendor Skinning Pipeline. Vendor-specific commands are aliased at parse-time into canonical engine queries. The unified engine returns abstract data structures which are dynamically serialized into hardware-accurate textual layouts.

When executing diagnostic commands (e.g., viewing routing tables or interface states), `netsim` parses vendor-specific commands (such as `display ip routing-table` for Huawei or `show router route-table` for Nokia) and dynamically maps them to canonical internal abstract engine operations.

Crucially, the resulting abstract protocol data is then serialized explicitly back into string layouts that strictly match real-world NOS hardware outputs. This allows operators to interactively debug the simulator or write automated text-based regex assertions directly against familiar hardware formats without the resource cost of booting gigabyte-sized vendor VM images.

Example: Batch simulation in a CI pipeline

```

# Run topology, output JSON, fail on assertion failures
netsim run topology.yaml --format json --output results.json

# Extract routing matrix from a running daemon
netsim routing-matrix my-fabric --output before.json

# Compare routing state before and after a change

```

```
netns diff before.json after.json --format ascii
```

9.3 Daemon Architecture

The daemon runs a simulation as a long-lived background process, enabling multiple users to attach, inspect, and inject commands concurrently. This transforms netsim from a batch-mode tool into an interactive, always-on simulation environment – akin to having a virtual lab where engineers can “walk up” to any device at any time.

9.3.1 Threading Model

Design Decision

Problem: The simulation engine is single-threaded and !Send (it holds non-thread-safe state such as protocol FSMs, FIBs with interior references, and mutable device vectors). But gRPC requires an async runtime. How to bridge the two?

Options considered:

1. **Async engine:** Rewrite the engine to be async-compatible. Invasive; every protocol state machine would need refactoring, and `.await` points would complicate the deterministic tick loop.
2. **Separate OS thread:** Run the engine on a dedicated OS thread; communicate via channels.
3. **Mutex wrapping:** Wrap the engine in `Arc<Mutex<Engine>>` and acquire the lock from async handlers. Would require the engine to be `Send`, and creates lock contention between gRPC handlers and the tick loop.

Decision: Separate OS thread. The engine runs its tick loop on a dedicated thread. A Tokio multi-threaded runtime on the main thread hosts the gRPC server. Communication uses `std::sync::mpsc` channels: commands flow from gRPC handlers to the engine thread, responses flow back via per-request `tokio::sync::mpsc` channels.

Trade-off: Requires channel-based indirection for every command. In practice, the overhead is negligible compared to tick processing time. The benefit is that the engine thread never blocks on I/O, and the gRPC server can handle arbitrary concurrent connections without starving the simulation.

The resulting architecture has three layers of concurrency:

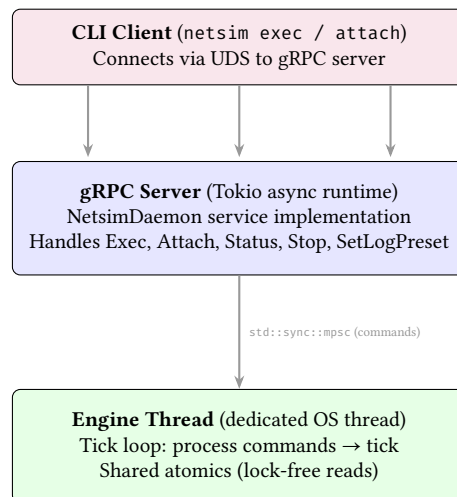


Figure 20. Daemon Concurrency Architecture. A Tokio-based gRPC server receives concurrent client connections and forwards commands to the strictly single-threaded Engine via channels.

Shared state between the gRPC server and the engine thread is communicated via two mechanisms:

1. **Command channel** (`std::sync::mpsc`): gRPC handlers send `EngineCommand` variants to the engine thread. Each command carries a per-request response channel for sending results back asynchronously.
2. **Atomic scalars** (`Arc<AtomicU64>`, `Arc<AtomicBool>`): The engine writes tick count, convergence status, and average tick duration every tick. The Status RPC reads these with `Ordering::Relaxed`, providing a lock-free path for metrics without blocking the tick loop.

9.3.2 Command Pipeline

The engine processes commands at tick boundaries, ensuring consistent state. Three command types flow from gRPC handlers into the engine thread, as shown in fig. 21.

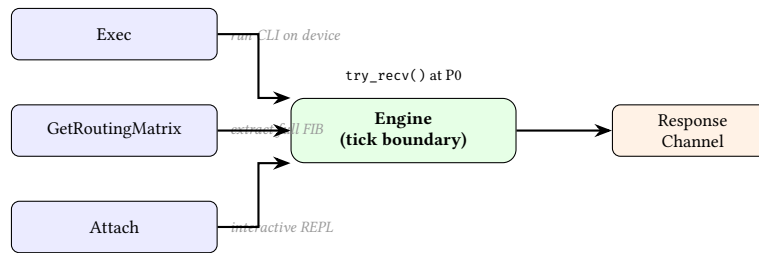


Figure 21. Command dispatch flow. gRPC handlers send one of three command variants to the engine via an MPSC channel. The engine drains the channel non-blockingly at the start of each tick (phase P0), ensuring commands always observe tick-boundary state. Each command carries a per-request response channel for asynchronous reply.

At the start of each tick (phase P0 of the tick pipeline), the engine calls `try_recv()` in a non-blocking loop to drain all pending commands. This guarantees three properties: commands never observe mid-tick intermediate state (they always see a tick boundary), the tick loop is never blocked waiting for commands (`try_recv()` returns immediately when no commands are pending), and simultaneous commands from multiple users produce consistent results because delivery is serialised within the single-threaded tick loop.

9.3.3 Interactive Attach Sessions

The Attach RPC creates a persistent bidirectional session between a CLI user and a simulated device. Unlike Exec (fire-and-forget), attach sessions persist across ticks:

1. The gRPC handler sends an `EngineCommand::Attach` with Tokio channel endpoints to the engine thread.
2. The engine stores the session in an internal `attach_sessions: Vec<AttachSession>`.
3. Every tick, `process_attach_sessions()` drains each session's `input_rx` for pending commands, executes them with full `&mut self` access, and sends results to `output_tx`.
4. Sessions are cleaned up when the user sends `exit/quit`, presses `Ctrl-D`, or the channel disconnects.

On the client side, the CLI runs a `rustyline`-based REPL on a blocking thread with full tab completion, command hints, and persistent history. This creates a workflow where a user can type IOS-style commands interactively while the simulation continues ticking in the background — commands are queued and executed at the next tick boundary.

9.3.4 Process Lifecycle

The `daemon start` command forks a background process using the `daemonize` crate. The startup sequence is carefully ordered:

1. **Parse CLI arguments:** Name, socket path, PID file, topology file, tick interval, log level.
2. **Daemonize:** Fork via `daemonize`, redirect `stdout/stderr` to log file, write PID file. This happens *before* building the Tokio runtime to isolate file descriptors.
3. **Load topology:** Parse YAML, build the Engine with `command_channel(engine.create_command_channel())`.
4. **Spawn engine thread:** Start the tick loop on a dedicated OS thread (the engine is `!Send`, so it must be created and used on the same thread).
5. **Start gRPC server:** Build Tokio multi-threaded runtime, bind to `.netsim/<name>.sock`, register `SIGTERM` handler for graceful shutdown.

The daemon writes three files to a `.netsim/` directory:

```
.netsim/
<name>.sock  # Unix Domain Socket for gRPC
<name>.pid   # Process ID file
<name>.log   # Structured log with file appender
```

Using Unix Domain Sockets (UDS) instead of TCP ports avoids port conflicts when running multiple daemons on the same machine, provides implicit access control via filesystem permissions, and eliminates the TCP overhead for local communication.

9.3.5 Daemon Management

The daemon lifecycle (`start`, `stop`, `status`, `list`, `restart`) is managed via the `daemon` subcommand family. Log verbosity can be changed at runtime via `daemon log <name> <preset>` without restarting the simula-

tion, choosing between `normal` (info), `protocol_debug` (OSPF/BGP/IS-IS/BFD at debug), or `full_trace` (all modules at trace level).

This is implemented via the `SetLogPreset` gRPC RPC, which reloads the tracing subscriber filter on-the-fly using tracing-subscriber's reload layer. Because the filter swap is atomic, there is no window where log events are dropped or duplicated.

9.4 gRPC Service Interface

The daemon exposes five RPCs via Protocol Buffers:

Table 5. gRPC service definition (`proto/netsim.proto`).

RPC	Type	Description
Exec	Server stream	Execute a command, stream output lines back
Attach	Bidi stream	Interactive REPL session with command/response streaming
Status	Unary	Return PID, uptime, tick count, convergence status, average tick duration, device list
Stop	Unary	Graceful shutdown
SetLogPreset	Unary	Change log verbosity at runtime

The `Status` RPC returns live metrics from shared atomics (tick counter, convergence flag, average tick duration), avoiding lock contention with the engine thread. The response includes a `DeviceInfo` array listing every device by name, type, and interfaces—enabling the TUI to build its device selection menu without parsing the topology file.

9.5 IOS-Style Command Tree

The interactive REPL implements a `BTreeMap`-based command vocabulary tree that provides tab completion and prefix matching. Partial commands are expanded to their unique completion (`sh ip ro` → `show ip route`); when a prefix is ambiguous (`sh i` matching `show interface`, `show ip`, and `show isis`), `netsim` reports an error with all matching completions. Interface names are completed dynamically based on the attached device's actual interfaces, and a grey hint shows the first available completion as the user types.

Design Decision

Why IOS-style CLI in a simulator?

Network engineers have decades of muscle memory for IOS commands. Making the simulator's interactive interface match that mental model reduces the learning curve to zero for the inspection workflow. An engineer who can type `show ip route` on a production router can type the same command in `netsim` and get comparable output.

The REPL supports over 50 commands spanning `show` (30+ variants covering every protocol), diagnostics (`ping`, `traceroute`, `dig`, `iperf`), runtime configuration (`route add`, `interface shutdown`), and session management. The full command reference is in section C.

9.6 Ping and Traceroute

The `ping` and `traceroute` commands are notable because they drive *re-entrant simulation stepping*. When a user types `ping 10.0.1.1`, `netsim` does not simply check reachability in the current routing table—it injects ICMP packets into the simulation and advances the tick loop until replies arrive or a timeout is reached.

`ping` sends ICMP Echo Requests from the attached host, advances the simulation tick by tick, collects Echo Replies, and reports sent/received/lost counts with min/max/avg RTT (default 4 probes, Host devices only). `traceroute` sends probes with incrementing TTL, collects Time Exceeded and Echo Reply messages, and displays hop-by-hop paths with latency (default max-TTL 30, up to 50,000 ticks per probe).

Note

Both commands support device hostname resolution in addition to IP addresses. `ping r1` resolves the hostname `r1` to its first interface IP via an internal device registry, providing a convenient shorthand.

9.7 Diagnostic Tools: dig and iperf

Two additional diagnostic commands extend the operator toolkit. `dig` resolves hostnames against a configured DNS server, reporting query time and response status codes (NOERROR, NXDOMAIN, timeout) via a state-machine implementation that drives simulation stepping. `iperf` generates constant-bit-rate traffic between two endpoints, reporting per-interval statistics (bytes sent, bits/second) and a total flow summary, with configurable duration, reporting interval, and packet size.

9.8 Static Analysis: Path Tracing, Diffing, and Capacity

`netsim` provides three static analysis tools for “Day 2” operations that analyse routing state without injecting packets. **Path tracing** computes hop-by-hop forwarding paths by walking the FIB/LFIB, identifying routing loops, blackholes, and asymmetric routing; traces handle complex overlays (MPLS, VXLAN) and recursive next-hops natively.

Topology diffing compares two routing matrices (e.g., before and after a planned configuration change) to quantify the blast radius, highlighting exactly which subnets shifted paths. This transforms abstract config reviews into concrete impact analyses:

Note

```
$ netsim diff before.yaml after.yaml --matrix
[WARN] Blast Radius Detected
- 10.0.1.0/24 (HQ Users): Path shifted (spine1 -> spine2)
- 192.168.1.0/24 (DB): Unreachable in after.yaml!
Summary: 1 subnet lost reachability, 1 subnet path shifted.
```

Capacity planning predicts congestion by overlaying a traffic matrix onto the current routing paths, flagging links that exceed their configured bandwidth.

Design Decision

Why static path tracing instead of packet-based traceroute?

Traceroute (above) injects real ICMP packets and advances the simulation — it shows the *actual* forwarding path including ECMP hash selection. Path tracing walks the FIB *without* advancing the simulation — it shows *all possible* paths and detects anomalies like loops and blackholes that traceroute might miss (a packet would simply be dropped). The two tools are complementary: traceroute for specific-flow validation, path tracing for exhaustive analysis.

9.9 JSON Structured Output

`netsim` provides structured JSON output at two levels. `netsim run --format json` emits a top-level result object containing simulation metadata (name, convergence tick, final tick), command outputs, assertion results, and any errors. At the command level, all 30+ `show` commands accept `--json` for structured output, enabling CI scripts to parse routing tables directly rather than screen-scraping ASCII.

9.10 Command Determinism

Commands received via gRPC are buffered and drained at phase P0 of the tick pipeline. This ensures that the simulation state observed by a command is always the state at a tick boundary, never a mid-tick intermediate state. Two users attaching simultaneously see consistent results because command delivery is serialised within the tick loop.

9.11 TUI Device Selector

Running `netsim` with no arguments launches a terminal user interface that presents a nested menu:

1. **Daemon selection:** Lists all running daemons (discovered from `.netsim/*.sock` files) with their status.
2. **Device selection:** Queries the selected daemon’s Status RPC for its device list, grouped by type (routers, hosts, switches).
3. **REPL:** Opens an interactive session on the selected device with full tab completion.

This provides a zero-configuration discovery workflow: start a daemon, then run `netsim` and navigate to any device without remembering daemon names or device identifiers.

9.12 Shell Completions & JSON Schema Emit

Two subcommands reduce friction in the edit-run-debug loop. `netsim completions <shell>` generates shell completion scripts for Bash, Zsh, Fish, PowerShell, and Elvish via `clap_complete`, covering all subcommands, flags, and argument values. `netsim schema` emits a JSON Schema for the full Topology struct

(derived via schemars), enabling real-time YAML validation and autocomplete in editors such as VS Code and Neovim with `yaml-language-server`.

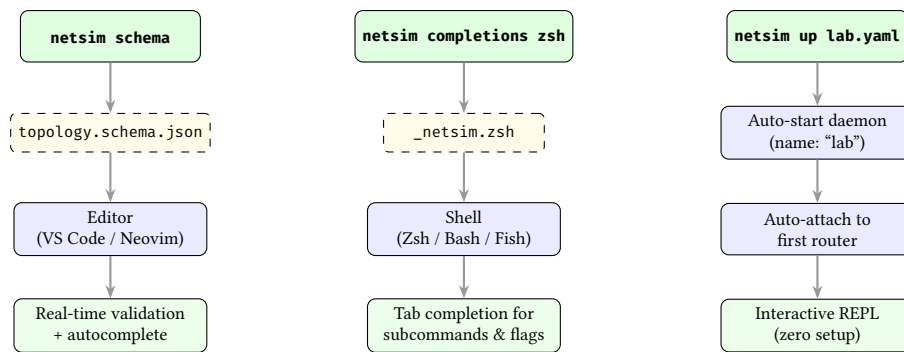


Figure 22. Developer Experience Shortcuts. Three zero-configuration workflows: JSON Schema for editor integration, shell completions for CLI discoverability, and `netsim up` for instant interactive sessions.

9.13 netsim up: Zero-Setup Interactive Sessions

The `netsim up <topology> [--node <name>]` command combines daemon startup and interactive attach into a single invocation. It derives the daemon name from the topology filename, parses the YAML to find the first router (or uses an explicit `--node`), starts the daemon at 100 ms tick interval, and drops the user directly into the IOS-like REPL:

Example: instant lab session

```

$ netsim up ospf-lab.yaml
Starting daemon 'ospf-lab'...
Attaching to r1...
r1# show ip route
 10.0.1.0/24 via 10.0.0.2 [OSPF, cost 20]
 10.0.2.0/24 directly connected, eth0
r1# exit

```

9.14 Host Shorthand & Bulk Script Commands

9.14.1 Host Shorthand

Topology files frequently define hosts with boilerplate: a single `eth0` interface, an IP address, and a default gateway. The `hosts:` shorthand reduces this to one line per host:

Listing 3. Host shorthand reduces boilerplate.

```

1 hosts:
2   - name: client1
3     ip: 10.0.1.10/24      # gateway inferred as .1
4   - name: client2
5     ip: 10.0.1.11/24
6     gateway: 10.0.1.254  # explicit override

```

Each entry auto-expands into a full device definition with a single `eth0` interface and a default gateway (inferred from the subnet unless explicitly specified).

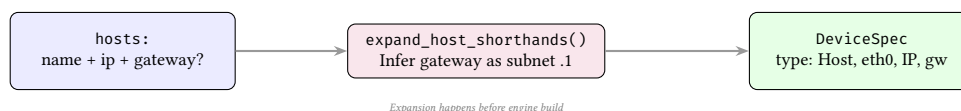


Figure 23. Host Shorthand Expansion. A minimal YAML declaration is expanded into a full `DeviceSpec` with interface, IP, and inferred gateway before the engine is constructed.

When gateway is omitted, the expansion logic takes the network address of the given subnet and sets the host portion to `.1` (e.g., `10.0.1.0/24` → gateway `10.0.1.1`). The expanded `DeviceSpec` entries are appended to `topology.devices` before engine construction.

9.14.2 Bulk Script Commands

The `script:` section supports a `devices:` list with macro selectors for running commands across device groups:

Listing 4. Bulk script commands with macro selectors.

```
1 script:
2   - at: converged
3     devices: [all_routers]
4     command: show ip route
5
6   - at: converged
7     devices: [r1, r2, r3]
8     command: show bgp summary
9
10  - at: converged
11    devices: [all_hosts]
12    command: ping 10.0.0.1 count 3
```

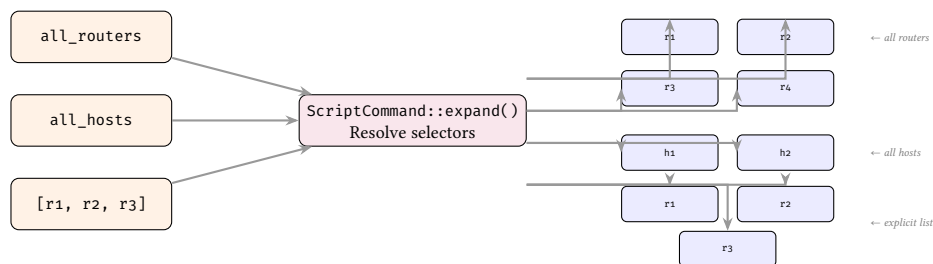


Figure 24. Bulk Script Expansion. Macro selectors (`all_routers`, `all_hosts`) are resolved against the topology's device list at parse time, producing one `ExpandedScriptCommand` per target device.

Each `ScriptCommand` with a `devices:` list is expanded into individual `ExpandedScriptCommand` entries, one per target device. The expansion uses `ScriptCommand::expand(&devices)` and happens before the engine is built, so the `--plan` preview reports accurate command counts.

9.15 Simulation Timeline & Event Log

The `--timeline` flag enables a per-tick event log that captures protocol state transitions during the simulation. After each `engine.tick()`, the simulation loop diffs current state against previous snapshots and emits `TimelineEvent` records for four event categories:

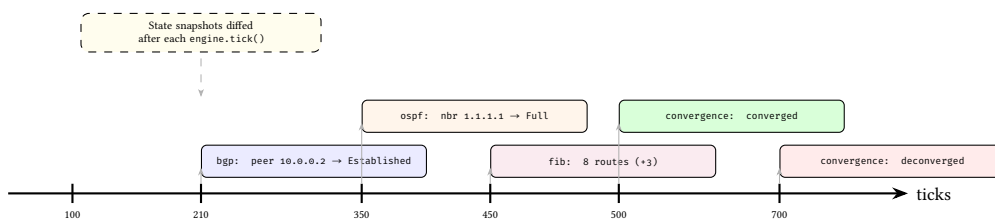


Figure 25. Timeline Event Collection. The simulation loop maintains per-device state snapshots for OSPF neighbors, BGP sessions, and FIB route counts. After each tick, diffs produce timestamped events that form a complete protocol convergence narrative.

Table 6. Timeline event categories and their triggers.

Category	Event Type	Trigger
ospf	state_change	OSPF neighbor transitions (e.g., Init → Full)
bgp	state_change	BGP session transitions (e.g., Active → Established)
fib	route_install	FIB route count change (reports delta: $+n$ or $-n$)
convergence	converged/deconverged	Network-wide convergence state change

The timeline is rendered in the ASCII summary (with aligned columns) and included in JSON output. When combined with the `--timeline` flag, operators can reconstruct exactly *when* each protocol transition occurred during convergence:

Example: timeline output

```
=== Timeline ===
[tick 210] r1      bgp      state_change: peer 10.0.0.2 -> Established
[tick 450] r2      ospf      state_change: neighbor 1.1.1.1 -> Full
[tick 451] r1      fib       route_install: 8 routes (+3)
[tick 500] *       convergence converged: network converged at tick 500
```

9.16 Plan Preview Mode

The `--plan` flag performs a static analysis of the topology file without constructing the engine or running any simulation ticks. This provides instant feedback on topology composition:

Example: plan preview

```
$ netsim run ospf-lab.yaml --plan
Topology: ospf-lab
Devices: 3 routers, 2 hosts (5 total)
Links: 4
Protocols: OSPF (area 0, 1), BGP (AS 65001, AS 65002)
Events: 2
Commands: 6 post-convergence
Assertions: 3
```

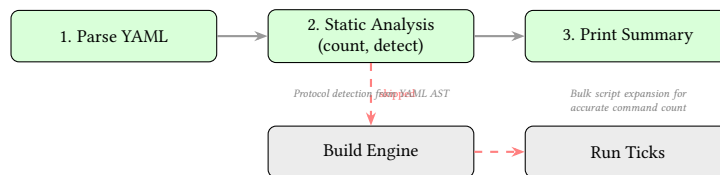


Figure 26. Plan Preview Pipeline. The `--plan` flag parses the YAML and performs static analysis (protocol detection, device counting, script expansion) without building the engine or running any simulation ticks.

Protocol detection operates on the YAML AST: OSPF is detected when any device or interface has an `ospf:` key (area IDs collected into a `BTreeSet`); BGP is detected from `bgp:` keys (ASNs collected); IS-IS from `isis:` keys. Script commands are expanded via `ScriptCommand::expand()` so bulk `all_routers` entries are counted accurately.

9.17 Contextual Error Messages

Build-time and runtime errors include contextual hints that suggest likely fixes, reducing the diagnosis loop from “error → read docs → guess → retry” to “error → fix”.

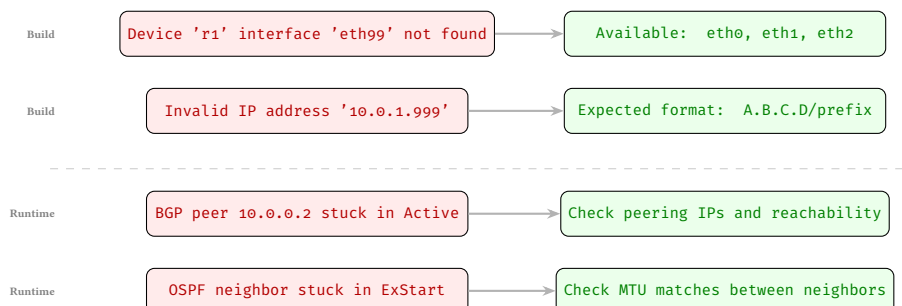


Figure 27. Contextual Error Hints. Build-time errors include available interface names; runtime convergence failures include protocol-specific diagnostic suggestions based on stuck FSM states.

Build-time hints. The `iface_not_found()` helper in the builder inspects the topology to list available interface names when a reference fails. The error message changes from the opaque “interface ‘eth99’ not found” to “interface ‘eth99’ not found. Available: eth0, eth1, eth2”.

Runtime convergence suggestions. After simulation, the `ConvergenceBreakdown` struct collects stuck protocol sessions and maps them to diagnostic suggestions via pattern matching on FSM states:

Table 7. Convergence diagnostic suggestion rules.

Protocol	Stuck State	Suggestion
BGP	Active	Check BGP peering IPs and reachability
BGP	OpenSent	Check AS numbers match expected iBGP/eBGP configuration
OSPF	ExStart	Check MTU matches between OSPF neighbors
OSPF	AreaMismatch	Check OSPF area ID configuration on both sides
(any)	(timeout)	Increase <code>--max-ticks</code> or check protocol configuration

Suggestions are printed to `stderr` with coloured “→” arrows after the summary banner:

Example: contextual convergence failure

```
X ospf-lab -- did not converge (timeout at tick 10000)
 5 devices | 0/4 commands ok | 0/3 assertions passed

Suggestions:
-> Check OSPF area ID configuration on both sides
   (r1 -> r2 stuck in AreaMismatch)
-> Increase --max-ticks or check protocol configuration
```

9.18 CI/CD Output Formats

Beyond the default ASCII output, `netnsim run` supports `--format json|junit|tap` for machine consumption. JUnit XML integrates natively with Jenkins, GitHub Actions, and GitLab CI; TAP v14 works with `prove` and `tap-parser`. All formats include a synthetic convergence test case plus one entry per assertion, enabling zero-script CI integration.

10 Deterministic Sandbox for Network AI and Automation

Current orchestration tools (Ansible, Nornir) and emerging agentic AI models require continuous, high-frequency feedback loops to validate actions against network infrastructure. Orchestrating these models against traditional virtualized hardware platforms (such as Containerlab or EVE-NG) incurs massive penalties: allocating management IP addresses, authenticating and negotiating SSH/NETCONF connection overhead, and waiting minutes for non-deterministic OS CPU scheduling delays before convergence occurs.

`netnsim` bypasses the management plane entirely, offering direct, instantaneous API and CLI execution against the simulation state in memory.

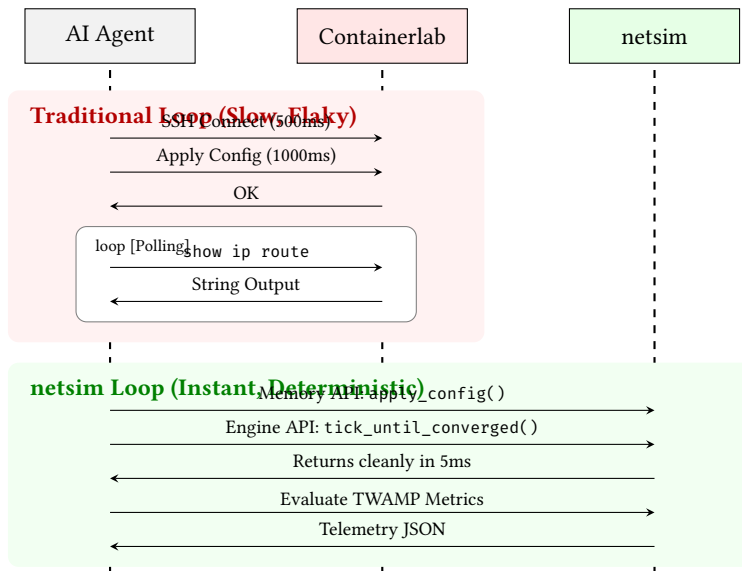


Figure 28. Comparison of the traditional orchestration feedback loop versus the netsim Sandbox in-memory workflow.

An LLM-based agent or reinforcement learning model can execute a configuration change, explicitly advance the simulation to the converged coordinate, and evaluate the structural outcome using advanced diagnostic telemetry. For instance, rather than parsing unstructured ping strings, agents can query TWAMP (Two-Way Active Measurement Protocol) results directly for p99 latency, jitter percentiles, and path trace structures across thousands of topological variations in seconds.

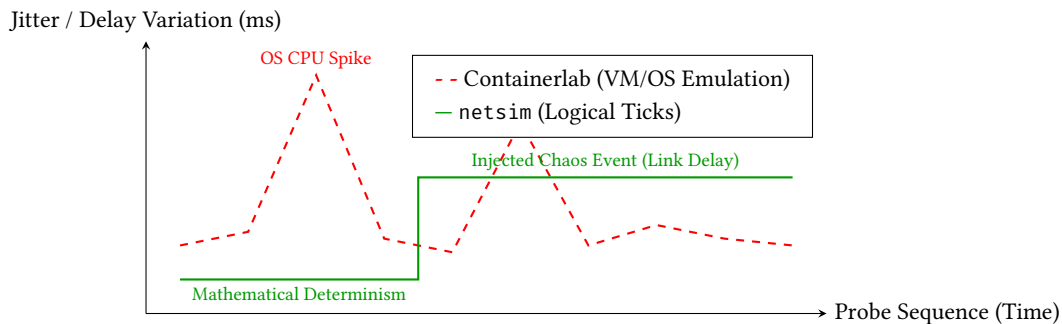


Figure 29. TWAMP Jitter Measurement: OS Emulation vs. Logical Tick Simulation. Traditional emulators suffer from non-deterministic host OS scheduling noise, severely polluting telemetry data. netsim provides perfectly flat, mathematically deterministic latency that only changes when explicitly instructed by the Chaos Scheduler.

This speed and mathematical determinism provide a high-fidelity “sandbox” environment that is structurally impossible to achieve with traditional VM-backed emulators.

11 Python Bindings and the Deterministic AI Sandbox

netsim provides native Python bindings via PyO3, which are explicitly designed to bypass SSH/NETCONF overhead, enabling the rapid, in-memory feedback loops required by the AI Agent Sandbox. By exposing the simulation engine directly to Python without serialisation overhead or network round-trips, Reinforcement Learning (RL) or LLM agents can execute configuration changes and instantly evaluate telemetry. This architecture specifically highlights advanced functional capabilities like **SRv6 Traffic Steering** and complex **Chaos Impairments** by allowing agents to inject and evaluate them in milliseconds.

Unlike the daemon mode (Section 9), which communicates via gRPC over Unix Domain Sockets, the Python bindings embed the Rust engine in-process — Python calls invoke Rust functions directly through the CPython C API.

Design Decision

Two integration paths: netsim offers two distinct ways to interact with the simulation programmatically:

1. **Daemon + gRPC:** A long-running background process with CLI or custom gRPC clients. Best for interactive exploration, multi-user access, and operational workflows.
2. **Python bindings (PyO3):** Direct in-process embedding. Best for scripted test automation, CI/CD pipelines, custom analysis, and integration with Python data science tooling (pandas, matplotlib, pytest).

Trade-off: The daemon supports concurrent multi-user access and persistent sessions. The Python bindings are single-threaded (matching the engine's !Send constraint) but offer zero-copy access to engine state and the full expressiveness of Python for orchestration.

11.1 AI Agent Integration Example

To illustrate the sandbox concept, consider a reinforcement learning (RL) agent tasked with optimising an SRv6 traffic engineering policy. Rather than waiting minutes for NETCONF commits and Containerlab convergence, the agent can update the configuration, advance the logical tick clock, and receive an immediate reward signal based on path tracing telemetry—all in the same process memory space.

Listing 5. Rapid in-memory feedback loop for an AI agent.

```

1 from netsim_py import Simulation
2
3 def step_environment(sim: Simulation, action: dict):
4     # 1. Agent applies policy change directly to memory
5     sim.run_command("r1", f"sr policy add {action['sid']} weight {action['w']}")
6
7     # 2. Advance logical time until protocols converge (typically < 10ms)
8     sim.run_until_converged(max_ticks=2000)
9
10    # 3. Extract zero-copy telemetry for reward calculation
11    trace = sim.run_command("r1", "traceroute 10.0.9.9 --json")
12    jitter = sim.run_command("r1", "show twamp statistics --json")
13
14    return calculate_reward(trace, jitter)

```

11.2 Architecture

The binding is implemented as a Rust crate (crates/netsim-py/) that compiles to a CPython extension module via maturin. The module exports six PyO3 classes:

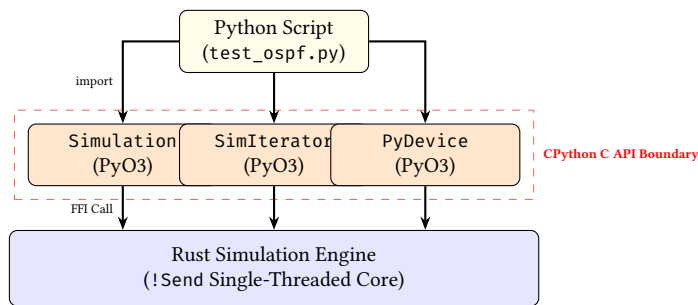


Figure 30. Python Bindings Architecture. The PyO3 wrappers expose the single-threaded Rust Engine directly to the Python interpreter, allowing zero-copy, in-process access to the simulation state.

Table 8. Python binding classes.

Class	Rust Wrapper	Purpose
Simulation	Engine	Topology construction, simulation control, event scheduling
SimulationIterator	Engine tick iterator	Pythonic for tick in <code>sim.run_iter()</code>
PyDevice	Device trait object	Per-device state access (counters, interfaces, routing)
RoutingTable	FIB accessor	Route lookup and enumeration
Counters	Interface counters	Packet/byte statistics (attribute and dict access)
InterfaceInfo	Interface state	Name, MAC, IP, up/down, per-interface counters

All classes that hold engine references are marked `#[pyclass(unsendable)]`, which tells PyO3 to enforce single-threaded access. This mirrors the engine's `!Send` constraint at the Python level — attempting to share a `Simulation` across Python threads raises a `RuntimeError`.

11.3 Topology Construction API

The `Simulation` class provides a fluent builder API with method chaining for ergonomic topology construction:

Listing 6. Building a triangle topology with OSPF.

```

1 from netsim_py import Simulation
2
3 with Simulation() as sim:
4     # Build topology with method chaining
5     sim.add_router("r1").add_router("r2").add_router("r3")
6
7     # Add interfaces with IP addressing
8     sim.add_interface("r1", "eth0", ip="10.0.12.1/24")
9     sim.add_interface("r1", "eth1", ip="10.0.13.1/24")
10    sim.add_interface("r2", "eth0", ip="10.0.12.2/24")
11    sim.add_interface("r2", "eth1", ip="10.0.23.2/24")
12    sim.add_interface("r3", "eth0", ip="10.0.13.3/24")
13    sim.add_interface("r3", "eth1", ip="10.0.23.3/24")
14
15    # Wire devices together
16    sim.add_link("r1", "r2", iface_a="eth0", iface_b="eth0")
17    sim.add_link("r1", "r3", iface_a="eth1", iface_b="eth0")
18    sim.add_link("r2", "r3", iface_a="eth1", iface_b="eth1")
19
20    # Enable OSPF (auto-assigns router ID, enables on all
21    # interfaces with IPs, default cost 10)
22    sim.enable_ospf("r1")
23    sim.enable_ospf("r2")
24    sim.enable_ospf("r3")
25
26    # Run until protocols converge
27    converged = sim.run_until_converged(max_ticks=5000)
28    print(f"Converged: {converged} at tick {sim.current_tick()}")

```

Hosts are created with optional IP and gateway configuration:

```

sim.add_host("h1", ip="10.0.1.100/24", gateway="10.0.1.1")
sim.add_host("h2", ip="10.0.2.100/24", gateway="10.0.2.1")
sim.add_link("h1", "r1") # auto-creates interfaces

```

11.4 Simulation Control

Four methods provide different levels of control over the tick loop:

Table 9. Simulation control methods.

Method	Behaviour
<code>sim.run(max_ticks)</code>	Run for exactly <code>max_ticks</code> ticks. Supports <code>on_progress</code> callback every 100 ticks.
<code>sim.run_until_converged()</code>	Run until <code>is_converged()</code> returns <code>True</code> or <code>max_ticks</code> reached. Returns <code>bool</code> .
<code>sim.step()</code>	Execute a single tick. Returns current tick number.
<code>sim.run_iter(max_ticks)</code>	Return a Python iterator that yields tick numbers. Supports <code>stop_on_converged=True</code> .

All methods honour Ctrl-C via `py.check_signals()`, raising `KeyboardInterrupt` cleanly without leaving the engine in an inconsistent state.

The iterator interface is particularly useful for custom convergence logic:

```
for tick in sim.run_iter(max_ticks=10000):
    if tick % 500 == 0:
        status = sim.convergence_status()
        print(f"Tick {tick}: OSPF={status['ospf_converged']}, "
              f"BGP={status['bgp_converged']}")
    if sim.is_converged():
        break
```

11.5 Device State Inspection

The `device()` method returns a `PyDevice` wrapper that provides live access to device state. Each property access reads fresh data from the engine — there is no stale caching:

Listing 7. Inspecting device state after convergence.

```
1 # Get device wrapper
2 r1 = sim.device("r1")
3
4 # Aggregate traffic counters
5 print(r1.counters)
6 # => Counters(packets_sent=1847, packets_received=1923, ...)
7
8 # Per-interface details
9 for iface in r1.interfaces:
10     print(f"{iface.name}: {iface.ip}/{iface.prefix_len} "
11           f"up={iface.is_up} tx={iface.counters.packets_sent}")
12
13 # Routing table access
14 rt = r1.routing_table
15 print(f"Routes: {len(rt)}")
16 for route in rt.routes:
17     print(f"  {route['prefix']} via {route['next_hop']} "
18           f"dev {route['interface']}")
19
20 # Specific route lookup
21 route = rt.lookup("10.0.23.0/24")
22 if route:
23     print(f"Next hop: {route['next_hop']}")
```

11.6 Event Scheduling

The Python API exposes the engine's event scheduler for chaos engineering and failure injection:

Listing 8. Scheduling link failure and recovery.

```
1 # Schedule link failure at tick 1000
2 fail_id = sim.schedule_event(1000, "interface_down",
3                               device="r1", interface="eth0")
4
5 # Schedule recovery 500 ticks later (relative)
6 sim.schedule_event_after(500, "interface_up",
7                           device="r1", interface="eth0")
8
9 # Dynamic topology changes
10 sim.schedule_event(2000, "remove_device",
11                   device="r3", mode="graceful")
12 sim.schedule_event(3000, "add_device",
```

```

13     name="r4", type="router")
14 sim.schedule_event(3001, "add_wire",
15     device_a="r1", device_b="r4")
16
17 # Cancel an event before it fires
18 sim.cancel_event(fail_id)
19 print(f"Pending events: {sim.pending_event_count()}")

```

Supported event actions: interface_down, interface_up, remove_device (with graceful or immediate mode), remove_wire, add_device, and add_wire.

11.7 Exception Hierarchy

The bindings define a structured exception hierarchy rooted at `NetsimError`. `DeviceNotFoundError` is raised when referencing a non-existent device by name or ID. `TopologyError` covers invalid topology operations such as duplicate device names or adding interfaces to hosts. `SimulationError` handles runtime errors like invalid event actions or missing arguments.

This enables idiomatic Python error handling:

```

from netsim_py import Simulation, DeviceNotFoundError

try:
    dev = sim.device("nonexistent")
except DeviceNotFoundError as e:
    print(f"Device not found: {e}")

```

11.8 Integration with pytest

The Python bindings integrate naturally with pytest for network validation test suites:

Listing 9. pytest integration for network validation.

```

1 import pytest
2 from netsim_py import Simulation
3
4 @pytest.fixture
5 def triangle_network():
6     """Build and converge a triangle OSPF topology."""
7     sim = Simulation()
8     sim.add_router("r1").add_router("r2").add_router("r3")
9     sim.add_interface("r1", "eth0", ip="10.0.12.1/24")
10    sim.add_interface("r2", "eth0", ip="10.0.12.2/24")
11    sim.add_link("r1", "r2", iface_a="eth0", iface_b="eth0")
12    sim.enable_ospf("r1").enable_ospf("r2").enable_ospf("r3")
13    sim.run_until_converged()
14    return sim
15
16 def test_ospf_full_mesh_reachability(triangle_network):
17     """All routers should have routes to all subnets."""
18     sim = triangle_network
19     r1_routes = sim.device("r1").routing_table.to_dict()
20     assert "10.0.12.0/24" in r1_routes
21
22 def test_link_failure_reconvergence(triangle_network):
23     """Traffic should reroute after a link failure."""
24     sim = triangle_network
25     sim.schedule_event(sim.current_tick() + 1,
26         "interface_down", device="r1", interface="eth0")
27     sim.run_until_converged(max_ticks=5000)
28     assert sim.is_converged()

```

12 Usage

The following examples progress from a minimal two-device reachability check to multi-protocol chaos engineering scenarios. Each example answers a specific operational question—the kind of question that would otherwise require a physical lab or container-based emulation to validate. Together they illustrate how netsim scales from a quick sanity check to a comprehensive pre-deployment validation suite.

12.1 Example 1: Simple Reachability Test

Listing 10. Minimal reachability test.

```

1 devices:
2 - name: r1
3   type: router

```

```

4   interfaces:
5     - name: eth0
6       ipv4: 10.0.1.1/24
7   - name: h1
8     type: host
9     interfaces:
10      - name: eth0
11        ipv4: 10.0.1.10/24
12        gateway: 10.0.1.1
13 links:
14   - endpoints: [r1:eth0, h1:eth0]
15 assertions:
16   - type: reachability
17     source: h1
18     destination: 10.0.1.1
19     expected: true

```

Question answered: “Can host h1 reach router r1?”

Run with: `netnsim run topology.yaml`. The output reports assertion results (pass/fail) and convergence timing.

12.2 Example 2: OSPF Reconvergence After Link Failure

Listing 11. OSPF reconvergence test (abbreviated).

```

1  # 5-router OSPF ring: r1--r2--r3--r4--r5--r1
2  devices:
3    - name: r1
4      type: router
5      # ... interfaces with OSPF area 0
6      # ... r2 through r5
7
8  events:
9    - at: converged + 100
10     action: link_down
11     link: r1-r2
12
13 assertions:
14   - type: reachability
15     source: h1
16     destination: h2
17     expected: true # via alternate path
18   - type: convergence_time
19     max_ticks: 200 # must reconverge within 200 ticks

```

Question answered: “Does traffic reroute after a link failure, and does it reconverge within 200 ticks?”

12.3 Example 3: BGP L3VPN with Post-Convergence Inspection

Listing 12. BGP VPN with convergence-relative scripting.

```

1  # PE/CE topology with VRFs and iBGP route reflection
2  script:
3    - at: converged + 500
4      device: CE1
5      command: ping 10.200.1.10
6    - at: converged + 1s
7      device: PE1
8      command: show bgp vpnv4 vrf CUSTOMER-A

```

Question answered: “Are VPN prefixes reachable after convergence? What does the VPNv4 table look like?”

12.4 Example 4: Chaos Engineering

Listing 13. Probabilistic failure with cascade rules.

```

1 failure_patterns:
2   - name: chaos-monkey
3     trigger: !probabilistic { probability: 0.1 }
4     selector: { by_tier: spine }
5     action: fail_random_link

```

```

6     recovery: !after_ticks { ticks: 50 }
7 cascade_rules:
8   - name: dual-failure-cascade
9     trigger: !dependency_based
10      primary_selector: { link_name_pattern: "spine-*" }
11      action: fail_devices

```

Question answered: “Does the fabric survive random spine failures with cascading effects?”

12.5 Example 5: VRRP Gateway Redundancy

Listing 14. VRRP first-hop redundancy with tracked uplinks (abbreviated).

```

1 vrrp:
2   groups:
3     - id: 1
4       virtual_ip: 10.0.1.1
5       routers:
6         - { device: r1, priority: 150, track_interface: eth1 }
7         - { device: r2, priority: 100 }
8   events:
9     - at: converged + 100
10      action: interface_down
11      device: r1
12      interface: eth1 # tracked uplink fails -> r2 takes over

```

Question answered: “When the primary gateway’s uplink fails, does VRRP failover to the backup and restore host reachability?”

12.6 Example 6: DHCP Relay Across Subnets

Listing 15. DHCP relay with Option 82 (abbreviated).

```

1 devices:
2   - name: r1
3     interfaces:
4       - { name: eth0, ipv4: "10.0.1.1/24",
5         helper_addresses: [10.0.100.10] }
6       - { name: eth1, ipv4: "10.0.100.1/24" }
7   - name: dhcp-server
8     type: host
9     interfaces:
10      - { name: eth0, ipv4: "10.0.100.10/24" }
11   - name: client
12     type: host # unconfigured, will obtain via DHCP

```

Question answered: “Does the relay agent correctly forward DHCP Discover packets from the client subnet to the server, populating GIADDR and Option 82?”

12.7 Example 7: SRv6 Traffic Steering

Listing 16. SRv6 traffic steering with explicit paths (abbreviated).

```

1 devices:
2   - name: ingress
3     srv6:
4       locator: "2001:db8:1::/48"
5       encap:
6         - dst: "10.0.3.0/24"
7         segments: ["2001:db8:2::1", "2001:db8:3::1"]
8   - name: transit
9     srv6:
10      sids: [{ sid: "2001:db8:2::1", behavior: end }]
11   - name: egress
12     srv6:
13      sids: [{ sid: "2001:db8:3::1", behavior: end.dt4 }]

```

Question answered: “Can traffic be steered through a specific transit node using IPv6 Segment Routing, and does the egress node correctly decapsulate and forward the inner IPv4 packet?”

12.8 Example 8: BGP Route Reflection

Listing 17. iBGP route reflector hierarchy.

```

1 devices:
2   - name: rr1
3     type: router
4     bgp:
5       asn: 65000
6       router_id: 1.1.1.1
7       cluster_id: 10.0.0.1
8       neighbors:
9         - peer: 10.0.0.2
10           route_reflector_client: true
11         - peer: 10.0.0.3
12           route_reflector_client: true
13   # ... leaves configured to peer with rr1

```

Question answered: “Do route reflectors correctly propagate routes to their clients without causing routing loops, respecting cluster_id and originator_id?”

12.9 Example 9: Multicast Group Membership

Listing 18. IGMP/PIM multicast with static RP (abbreviated).

```

1 # RP router (10.0.0.1), leaf router, receiver host
2 # OSPF provides unicast reachability for RPF
3 devices:
4   - name: rp
5     type: router
6     multicast:
7       rp: { "239.0.0.0/8": 10.0.0.1 }
8   - name: receiver
9     type: host
10    multicast_groups: [239.1.1.1]
11    igmp_report_interval_ticks: 5000
12 links:
13   - endpoints: ["rp:eth0", "leaf:eth0"]
14   - endpoints: ["leaf:eth1", "receiver:eth0"]

```

Question answered: “Does the receiver’s IGMP join propagate through the PIM-SM control plane, creating a valid MFIB entry with correct RPF state toward the RP?”

13 Example Topologies

netsim ships with over 50 example topologies, embedded in the CLI binary via `include_str!()`. They span four orders of magnitude in scale (fig. 31), from a minimal 2-device link to a 486-device DC fabric, and serve three purposes: onboarding (learning the YAML format), regression testing (every example is gated by a convergence test), and benchmarking (the larger examples exercise adaptive parallelism).

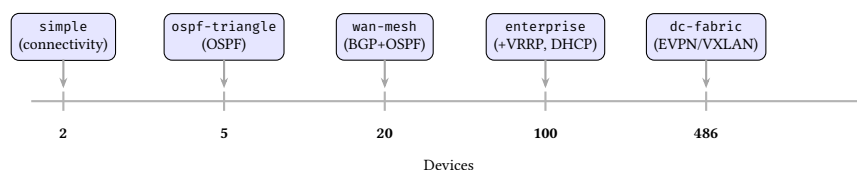


Figure 31. Topology scale spectrum. The example suite spans from 2-device connectivity tests to a 486-device DC fabric, with increasing protocol complexity at each scale tier.

13.1 Built-in Examples (`netsim example`)

The `netsim example <name>` subcommand prints a complete, runnable topology to stdout. Eight examples are available:

Table 10. Built-in example topologies.

Name	Protocols	Devices	Demonstrates
simple	—	2	Minimal connectivity
ospf-triangle	OSPF	5	Basic IGP routing
data-center	OSPF	20	Spine-leaf L3 fabric
wan-mesh	OSPF	18	Full-mesh WAN with costs
enterprise-campus	OSPF	18	3-tier campus hierarchy
mixed-network	OSPF	15	Router/Switch/Hub mix
large-enterprise	OSPF, Traffic	440	Multi-area enterprise
service-provider	OSPF, Traffic	108	P/PE/CE hierarchy

Example: Generate and run a topology

```
# Print the service-provider example
netsim example service-provider > sp.yaml

# Run it
netsim run sp.yaml --format json --output sp-results.json
```

13.2 Extended Examples

The examples/ directory contains 50+ topologies covering specialised protocols and advanced scenarios. Table 11 highlights the most interesting ones, grouped by the protocol stack they exercise.

Table 11. Selected extended example topologies.

Example	Protocols	Dev.	Demonstrates
<i>IGP & Link-State Routing</i>			
isis-hierarchical	IS-IS	8	L1/L2/L1L2 areas
isis-lan	IS-IS	4	Broadcast LAN DIS election
scale-test-ring	OSPF	100	100-device ring convergence
<i>BGP & Policy</i>			
bgp-ipv6-multi-as	BGP	3	Multi-AS IPv6 peering
bgp-community-policy	OSPF, BGP	3	NO_EXPORT, NO_ADVERTISE
docs-bgp-rr-loop	BGP	3	iBGP route reflection loops
<i>MPLS, SR & Tunneling</i>			
docs-mpls-ldp-oam	OSPF, MPLS/LDP	5	LSP ping/traceroute
rsvp-te-tunnel	OSPF, RSVP-TE	4	Explicit path tunnels
sr-mpls-transport	OSPF, SR-MPLS	6	Segment Routing with SRGB
gre-overlay	OSPF, GRE	5	GRE over OSPF underlay
<i>L3VPN & Data Centre</i>			
l3vpn-service-provider	OSPF, BGP, MPLS	7	VRF, VPNv4, PE/CE
dc-fabric-irb	OSPF, BGP, VXLAN	8	Integrated routing/bridging
dc-fabric-multi-homing	BGP	10	EVPN multi-homing
dual-stack-data-center	OSPF	20	IPv4 + IPv6 fabric
<i>Enterprise & Campus</i>			
vrrp-redundancy	VRRP	3	First-hop gateway failover
dhcp-relay-simple	DHCP Relay	3	Broadcast-to-unicast relay
rip-simple-network	RIP	4	Distance-vector convergence
<i>Fast Failover & Infrastructure</i>			

Example	Protocols	Dev.	Demonstrates
bfd-fast-failover	OSPF, BGP, BFD	3	Sub-second BFD failover
lag-lacp	LACP	4	Dynamic LAG negotiation
<i>Traffic & Chaos</i>			
dc-fabric-traffic	OSPF, Traffic	10	Traffic generation in DC
qos-congestion	OSPF, Traffic	4	Congestion & queue drops
chaos-simple	—	6	Failure injection patterns
datacenter-disaster	—	10	Multi-failure disaster
jitter-validation	—	6	Impairment validation
<i>Dual-Stack & Scale</i>			
dual-stack-service-provider	OSPF, Traffic	108	IPv4+IPv6 SP
large-enterprise	OSPF, Traffic	440	Multi-area enterprise

Note

Every example in this table is covered by an automated regression test in `tests/example_topology_regression.rs`. Each test verifies that the topology parses, builds, and converges within 50,000 ticks with a convergence threshold of 500 stable ticks. This ensures that protocol changes never silently break existing topologies.

Every protocol has at least one end-to-end topology test in the example suite, ensuring full coverage across all 17 protocol families.

14 Case Study: Validating Cascading Protocol Failures (Facebook 2021 Outage)

To demonstrate `netsim`'s value as a pre-deployment validation tool, we focus on validating cascading protocol failures—scenarios where a localized configuration error or link failure triggers a complex sequence of control-plane events across multiple protocols. To ground this in a real-world scenario, we model the October 2021 Facebook outage, where a routine maintenance command inadvertently severed the connections between Facebook's backbone routers and its data centres. Because the backbone routers could no longer reach the DNS servers, BGP automatically withdrew the routes for Facebook's DNS prefixes from the global internet.

Basic static configuration checks might easily miss this emergent, dynamic behaviour, seeing the link shutdown as a syntactically valid operation. By contrast, a dynamic simulator like `netsim` acts as the missing primitive in infrastructure CI/CD, providing software-engineering-grade testability for network operations.

14.1 Modeling the Scenario in `netsim`

We model a simplified topology representing this dynamic, containing a global transit provider, Facebook backbone routers, and a data centre housing the authoritative DNS servers for `185.89.218.0/24`.

Listing 19. Dynamic validation of the Facebook 2021 outage (abbreviated).

```

1 devices:
2   - { name: transit-provider, bgp: { as: 3356 } }
3   - { name: fb-backbone-1, bgp: { as: 32934 } }
4   - name: fb-dc-1
5     bgp:
6       as: 32934
7       networks: ["185.89.218.0/24"] # Authoritative DNS prefix
8 links:
9   - endpoints: [transit-provider:eth1, fb-backbone-1:eth1] # eBGP
10  - endpoints: [fb-backbone-1:eth2, fb-dc-1:eth1] # OSPF + iBGP
11 events:
12   - at: 500
13     interface_down:
14       device: fb-backbone-1
15       interface: eth2 # Fatal maintenance command
16 assertions:
17   - type: reachability
18     source: internet-user

```

```

19 destination: 185.89.218.12
20 expected: false # DNS globally unreachable

```

14.2 The Cascading Failure State Machine

When the simulated event triggers, netsim models the temporal propagation of the failure across protocol layers. Figure 32 visualizes the sequence of state transitions that culminate in the global BGP withdrawal.

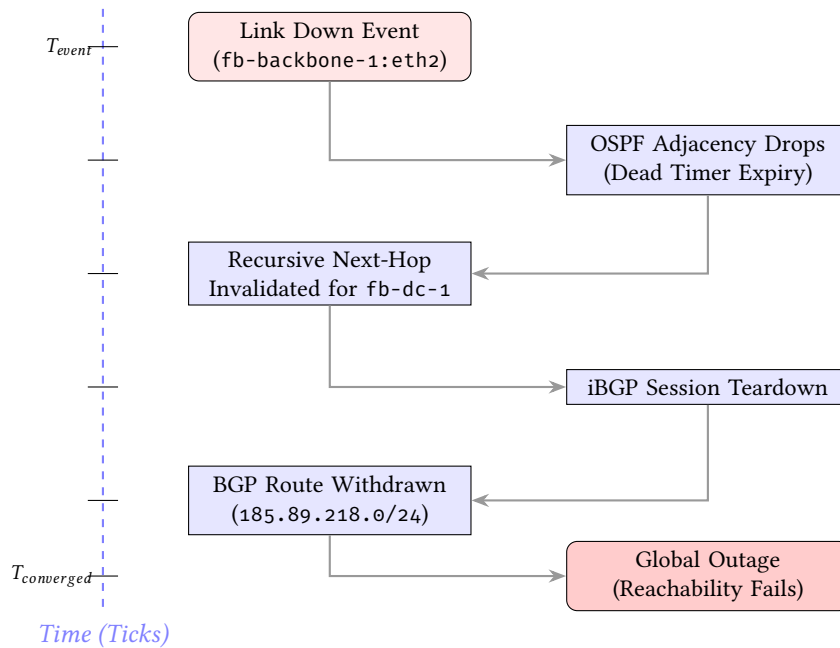


Figure 32. Cascading Protocol Failure Sequence. The simulator executes the complex state transitions resulting from a single physical layer event, demonstrating the precise temporal dimension missing in static configuration validation.

14.3 Execution and CI/CD Impact

When executed, netsim tracks the state deterministically: 1. At $t = 0$, BGP sessions establish and the 185.89.218.0/24 prefix is advertised from the data centre to the backbone, and out to the internet core. 2. At tick 500, the maintenance event triggers, bringing down the backbone-to-DC links. 3. The engine drops the OSPF adjacency, invalidating the recursive next-hop required for the iBGP session to the data centre routers. 4. fb-backbone-1 removes the prefix from its Loc-RIB and queues a BGP WITHDRAW message to the transit-provider. 5. The reachability assertion is evaluated after convergence. The simulator correctly flags that the internet user can no longer reach the authoritative DNS servers.

Because netsim abstracts away heavy OS dependencies and converges in milliseconds, this dynamic validation can run inline as a continuous integration check. It transforms “what-if” operational questions into fast, deterministic, and automatable tests, ensuring complex protocol interactions behave as intended before reaching production.

15 Case Study: Data Centre Fabric Validation (EVPN/VXLAN)

Beyond single-protocol cascading failures, the defining challenge of modern network testing is validating overlay fabrics. A typical Data Centre (DC) design uses a Clos topology (leaf-spine) with an OSPF or eBGP underlay, and a BGP EVPN overlay driving VXLAN encapsulation.

Validating a 5-stage Clos network with EVPN Type-2 (MAC/IP) and Type-5 (IP Prefix) routes, Anycast Gateways, and multi-homed servers (LACP/ESI) usually requires massive Containerlab deployments. Booting 20 FRR or EOS containers consumes tens of gigabytes of RAM and takes several minutes before the control plane even begins to converge, making it unsuitable for rapid CI/CD gates.

15.1 Modeling the Fabric in netsim

netsim models this entire multi-homed fabric natively in a single YAML file, executing the full EVPN route-type matrix without any underlying kernel namespaces.

Listing 20. EVPN/VXLAN Data Centre validation (abbreviated).

```

1 devices:
2   - name: spine1

```



```

3   bgp: { asn: 65000, evpn_rr: true }
4   - name: leaf1
5     bgp:
6       asn: 65001
7       evpn:
8         vnis:
9           - { vni: 10100, vrf: cust-a, type: l3 }
10          - { vni: 10110, vlan: 10, type: l2 }
11     interfaces:
12       - name: eth1
13         evpn_esi: "00:11:22:33:44:55:66:77:88:99" # Multi-homing
14 assertions:
15   - type: reachability
16     source: server1
17     destination: server2
18     expected: true
19   - type: path
20     source: server1
21     destination: server2
22     # Ensure traffic hashes across both spines via VXLAN UDP ECMP
23     expected_hops: [{ device: spine1 }, { device: spine2 }]

```

15.2 Execution and Validation Impact

When executed, netsim's lock-free engine simulates the entire fabric convergence: 1. The OSPF underlay converges, establishing loopback reachability. 2. iBGP EVPN sessions establish between the leafs and the spine Route Reflectors. 3. The servers initiate LACP bonds to the multi-homed leafs. 4. The leafs generate EVPN Type-4 (Ethernet Segment) routes to elect a Designated Forwarder, followed by Type-1 (Auto-Discovery) routes for aliasing. 5. As servers send ARP requests, the leafs intercept them, populate their local MAC VRFs, and advertise EVPN Type-2 (MAC/IP) routes across the fabric. 6. The assertion engine fires, validating that the overlay paths are correctly load-balanced via ECMP across the spines.

This entire sequence—from cold boot to EVPN overlay convergence and multipath validation—completes in under 50 milliseconds on standard laptop hardware. This fundamentally changes the economics of fabric testing, allowing engineers to diff topology changes (e.g., adding a new VRF or modifying an ESI) and receive instant, deterministic feedback on the blast radius before deploying to production.

16 Case Study: Service Provider L3VPN with Overlapping IP Space

A defining characteristic of Service Provider architectures is the ability to securely isolate multiple customer routing domains over a shared MPLS backbone. netsim natively supports this via Virtual Routing and Forwarding (VRF) instances, BGP VPNv4 address families, and MPLS label stacking.

Consider a scenario where two independent enterprise customers (Customer A and Customer B) use the exact same private RFC 1918 IP address space (e.g., 10.1.1.0/24). They both connect to the same Provider Edge (PE) routers. Validating this setup requires proving strict isolation: traffic from Customer A must never bleed into Customer B's network, even when the destination IPs are identical.

16.1 Modeling VRFs and Route Targets

We configure the PE routers with distinct VRFs for each customer, isolating their routing tables. BGP VPNv4 uses Route Distinguishers (RD) to make the overlapping prefixes globally unique within the provider core, and Route Targets (RT) to control import/export policies between the VRFs.

Listing 21. MPLS L3VPN isolation validation (abbreviated).

```

1 devices:
2   - name: pe1
3     vrfs:
4       - name: cust-a
5         bgp:
6           rd: "65000:100"
7           export_rt: ["65000:100"]
8           import_rt: ["65000:100"]
9       - name: cust-b
10        bgp:
11          rd: "65000:200"
12          export_rt: ["65000:200"]
13          import_rt: ["65000:200"]
14 assertions:

```

```

15 # Customer A can reach its own remote site
16 - type: reachability
17   source: ce1-cust-a
18   destination: 10.1.1.1 # Customer A's remote site
19   expected: true
20
21 # Crucially: Customer A CANNOT reach Customer B's site
22 - type: reachability
23   source: ce1-cust-a
24   destination: 10.1.1.1 # Same IP, but Customer B's site
25   expected: false

```

16.2 Execution and Validation Impact

During the simulation tick pipeline, `netsim` accurately models the complex control plane interactions: 1. The IGP (OSPF/IS-IS) underlay converges, and LDP distributes MPLS transport labels. 2. The BGP VPNv4 session establishes between `pe1` and `pe2`. 3. Customer A's `10.1.1.0/24` prefix is learned by `pe1`, converted to a VPNv4 route (`65000:100:10.1.1.0/24`), attached to the `65000:100` Route Target, assigned a VPN label, and advertised to `pe2`. 4. `pe2` receives the route, matches the Route Target against Customer A's VRF import policy, and installs it in the `cust-a` local routing table.

When the reachability assertions are evaluated, `netsim` walks the multi-label forwarding path (Transport Label + VPN Label). It definitively proves that because Customer B uses a different Route Target (`65000:200`), its traffic remains strictly isolated. This allows Service Providers to mathematically guarantee tenant isolation policies in their CI/CD pipelines before provisioning new customer VRFs on live edge routers.

17 Case Study: Automated Remediation via Python AI Agent

The convergence of AI agents and network operations promises "self-healing" networks, where agents autonomously detect anomalies and apply remediations. However, training and validating these agents against live infrastructure or slow OS-level emulators is impractical due to the high penalty of failure and slow feedback loops.

`netsim` natively solves this through its zero-overhead PyO3 Python bindings (section 11), allowing an external Python agent to interactively build topologies, manipulate state, and poll the Forwarding Information Base (FIB) in memory.

17.1 The Agent Control Loop

Consider a simple Python agent tasked with maintaining reachability across a 3-router ring. It monitors the primary path and, if a failure occurs, waits for OSPF to reconverge. If the primary path remains down, it automatically injects a remediation command to restore the physical link.

Listing 22. An autonomous self-healing script polling the `netsim` engine.

```

1 from netsim_py import Simulation
2
3 with Simulation() as sim:
4     # 1. Build topology & advance time until OSPF converges
5     sim.add_router("r1").add_router("r2").add_router("r3")
6     sim.run_until_converged(max_ticks=5000)
7
8     # 2. Inject a fatal physical layer failure
9     sim.schedule_event(sim.current_tick() + 10, "interface_down",
10                        device="r1", interface="eth12")
11
12     # 3. The Agent Loop: Monitor FIB reachability every 100 ticks
13     rt1 = sim.device("r1").routing_table
14     for i in range(50):
15         sim.run(max_ticks=100)
16         route = rt1.lookup("2.2.2.2/32")
17         if route and route['interface'] == "eth31":
18             print("ALERT: Primary link failed. Traffic rerouted via R3.")
19             break
20
21     # 4. Auto-Remediation: Agent restores the physical link
22     sim.schedule_event(sim.current_tick() + 10, "interface_up",
23                        device="r1", interface="eth12")
24     sim.run_until_converged(max_ticks=5000)

```

17.2 Execution and Validation Impact

Because the Python script embeds the Rust Engine directly within the same process space, there is zero network serialization or SSH overhead. The agent can poll the `routing_table` structure tens of thousands of times per second.

When executed, the agent immediately detects that the `2.2.2.2/32` route has disappeared from the `eth12` outbound interface and shifted to the `eth31` backup path following OSPF reconvergence. The script then successfully schedules the remediation event, proving that complex, stateful RL or LLM-based autonomous agents can use `netsim` to safely explore “chaos” scenarios and validate their logic millions of times faster than real-time.

18 Speculative Remediation: Monte Carlo Tree Search for Networks

The ultimate evolution of the `netsim` AI sandbox is *Speculative State-Space Branching*. Because the engine is deterministic and the network state is represented as pure Rust data structures, the simulation can be treated as a version-controlled object.

When an anomaly is detected (e.g., a link failure or BGP hijack), traditional automation is forced to make a single, high-stakes decision on live infrastructure. `netsim` enables a radically different approach: **speculative parallel execution**.

18.1 The Multi-Universe Remediation Loop

As shown in fig. 33, the agent can fork the current simulation state into multiple independent “universes” in memory. Each fork applies a different speculative patch:

1. **Fork A:** Redirect traffic via a backup GRE tunnel.
2. **Fork B:** Adjust BGP Local Preference to prefer an alternate peer.
3. **Fork C:** Implement strict inbound prefix filtering.

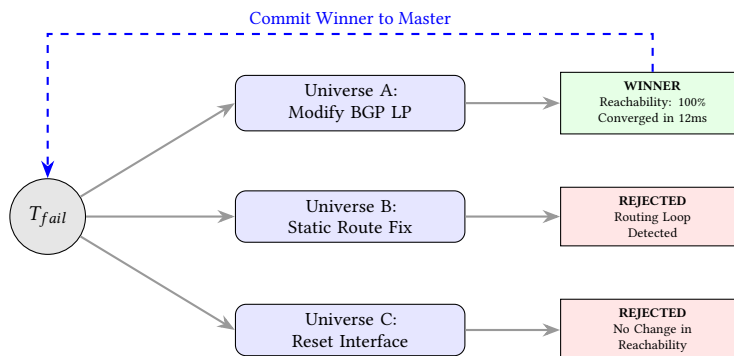


Figure 33. Speculative State-Space Branching. The engine forks the failure state into parallel universes, executes all candidate remediations to convergence, and commits only the optimal solution back to the master simulation.

18.2 Architectural Enablement

This capability is structurally impossible for OS-level emulators (Containerlab, GNS3) because snapshotting and forking 20 Linux kernels and their filesystem state is prohibitively slow and resource-heavy. In `netsim`, forking a universe is a zero-copy memory operation (cloning the Engine struct).

By applying Monte Carlo Tree Search (MCTS) principles to the network control plane, `netsim` transforms network operations from a game of guesswork into a solved search problem. The agent explores the entire state space of possible remediations in milliseconds, mathematically guaranteeing the outcome before a single command is ever sent to a production device.

19 Deterministic Blast Radius Explorer

Building on speculative branching, `netsim` enables an automated “Red Team” mode for network resilience: the *Deterministic Blast Radius Explorer*. Rather than waiting for a failure to occur, the engine proactively brute-forces every single point of failure (SPOF) in the topology to map the global fragility of the control plane.

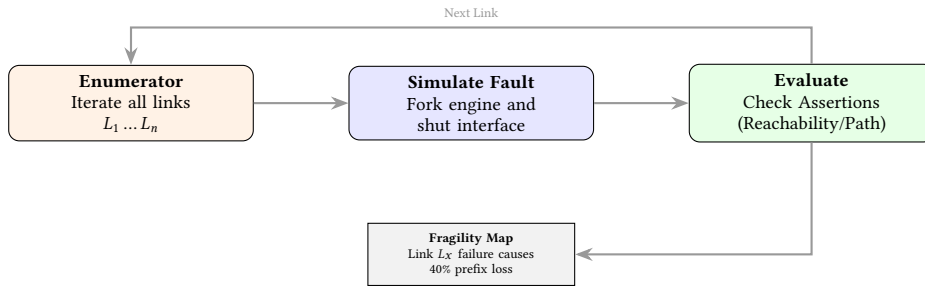


Figure 34. The Blast Radius Explorer Loop. The engine automatically iterates through every topological link, simulating a failure and recording the resulting prefix loss. This generates a deterministic heatmap of network fragility in seconds.

The Explorer automates the “what-if” analysis that would take hours of manual work in traditional systems. For a 100-link topology, the engine can execute 100 parallel failure scenarios, each converging in 10ms, completing a total network resilience audit in approximately one second. The output is a *Fragility Map*, highlighting critical links where a single failure would violate SLA reachability constraints.

20 The AI Network Doctor: LLM-Augmented Diagnostics

While *netsim* provides structured JSON telemetry, identifying the root cause of a complex convergence failure still requires human expertise. The *AI Network Doctor* bridges this gap by piping the entire zero-copy simulator state directly into a Large Language Model (LLM).

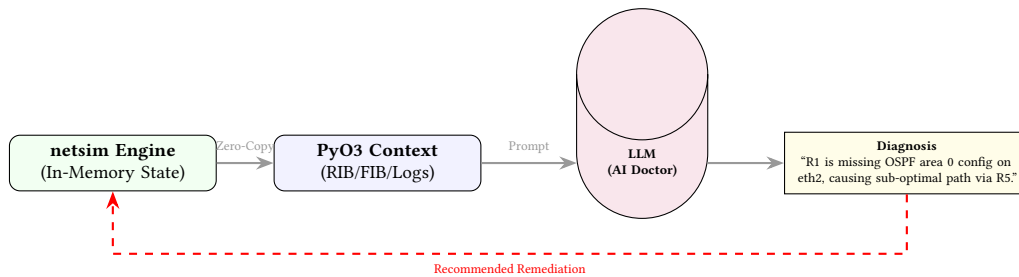


Figure 35. The AI Network Doctor Diagnostic Loop. The simulator state is exported as a context-rich prompt to an LLM, which returns a natural-language diagnosis and a set of corrective commands.

Unlike traditional “AI for Networks” which relies on noisy, sampled SNMP/Syslog data from production, the AI Doctor has access to the **perfect ground truth** of the simulator. It can see every OSPF LSA, every BGP attribute, and every packet drop. This enables “human-in-the-loop” automation where an operator can ask: *netsim* ask “Why is traffic to 10.0.9.9 taking the slow path?”. The engine analyzes the SPF tree, checks BGP local-pref, and returns a verified explanation.

21 Digital Twin Ingestion: From Production to Simulation

To maximize the accretive value of *netsim* in production environments, we developed an automated *Digital Twin Ingestion Pipeline*. This system bridges the gap between heterogeneous vendor configurations (Cisco, Juniper, Arista, Nokia) and the declarative YAML topology required by the simulator.

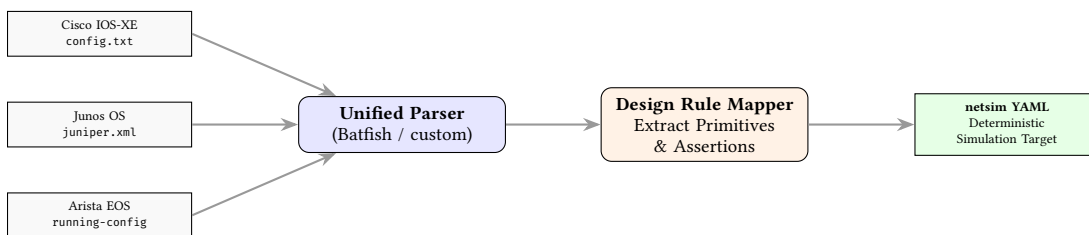


Figure 36. The Digital Twin Ingestion Pipeline. Heterogeneous vendor configurations are parsed and mapped into a standardized logical blueprint, which is then serialized as a *netsim* YAML topology for high-speed validation.

The pipeline performs deep inspection of the production configurations to extract the “visual intent” of the architecture. It identifies OSPF areas, BGP peering meshes, and VLAN boundaries, mapping them to the

engine’s internal protocol primitives. This allows a network operator to snapshot their entire global backbone and instantiate a millisecond-scale digital twin on a single laptop, providing a safe sandbox for “pre-fighting” configuration changes before they are committed to live hardware.

22 System Architecture

netsim processes a simulation in three stages: *build*, *run*, and *output*.

22.1 Build Stage

The Builder reads a YAML topology file and constructs the in-memory simulation state:

1. **Schema validation:** The YAML is validated against the topology schema. Missing required fields, unknown device types, and invalid IP addresses are caught here.
2. **Device construction:** Each device entry creates a Router, Host, Switch, or Hub with configured interfaces.
3. **Wire creation:** Each link entry creates a Wire connecting two interfaces. Latency, loss, and jitter are applied from named impairment profiles or inline values.
4. **Pattern resolution:** Pattern-based selectors (`by_tier: spine, name_pattern: "r*"`) are expanded to concrete device/link lists.
5. **Event scheduling:** Failure patterns (`FailurePattern`) are expanded into a sequence of `ScheduledEvents`. Convergence-relative events (`at: converged + 500ms`) are deferred until convergence is detected at runtime.

Design Decision

Why separate build from run?

Separating build from run allows the Builder to perform expensive validation (schema checks, pattern expansion, event scheduling) once, producing an immutable simulation configuration. The engine then runs without re-validating on every tick. This also enables future features like topology caching and incremental rebuilds.

22.2 Run Stage: The Tick Loop

The engine advances simulation time one tick at a time (default: 1 ms/tick). It uses a **Zero-Copy Structure-of-Arrays (SoA)** architecture to maximise parallel execution performance and CPU cache hit rates. Device storage uses stable memory locations (SlotMaps), allowing pointers and indices to persist across ticks without the $O(N)$ reconstruction overhead typical of object-oriented designs.

Each tick executes eight phases in strict order:

Table 12. The 8-phase tick pipeline.

Phase	Name	Description
P0	Drain commands	Dequeue pending gRPC commands for deterministic delivery
P1	Device→Wire	Move outbound frames from device output queues to connected wires
P2	Tick devices	Run all protocol state machines (OSPF SPF, BGP decision, BFD timers, etc.). Parallel via Rayon if ≥ 500 devices
P2.5	EVPN sync	Snapshot-then-apply EVPN Loc-RIB synchronisation
P3	Tick wires	Advance in-transit frames by one tick; apply impairments (loss, jitter)
P4	Wire→Device	Deliver arrived frames to destination device input queues
P5	Convergence check	Evaluate the composite convergence predicate (section 23)
P6	Hooks & quiescence	Check quiescence; fire convergence hooks; schedule deferred events
P7	Export	Write PCAP, BMP, NetFlow snapshots

Note

The phase ordering is load-bearing. P1 must precede P2 so that devices process frames that were sent in the *previous* tick, not the current one. P2.5 must follow P2 so that EVPN updates reflect the current tick's BGP processing. Reordering phases will break protocol correctness.

22.3 Rust Ownership and Data Flow

To achieve lock-free parallel execution in a single tick loop, netsim relies on a bipartite borrow model. The overarching `Simulation` struct splits its state into disjoint mutable borrows—specifically `&mut devices` and `&mut wires`—allowing the engine to process them concurrently without runtime locking overhead.

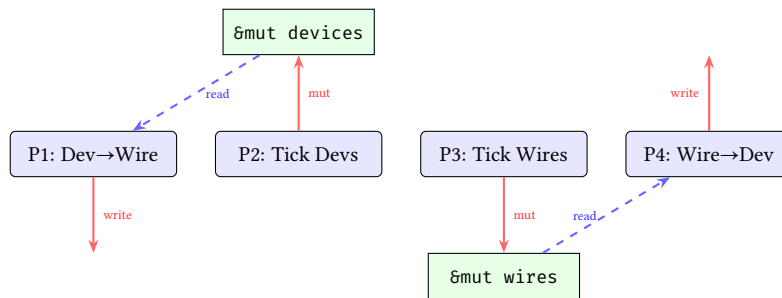


Figure 37. The 8-Phase Memory Ownership Pipeline. Disjoint mutable borrows of devices and wires allow lock-free parallelism across the tick loop. Colored arrows indicate read/write boundaries.

During Phase 2 (Tick devices), `devices` is exclusively borrowed mutably, allowing `par_iter_mut` to evaluate all protocol state machines concurrently. Wires are left untouched. In Phase 3, `wires` is mutably borrowed to advance packets in transit. Phases 1 and 4 handle the synchronization boundaries between devices and wires, ensuring strict, deterministic data flow.

22.4 Determinism Guarantees

Three properties ensure that two runs of the same topology produce identical results:

1. **Integer ticks:** All protocol timers are expressed in integer ticks. No floating-point arithmetic appears in timing-sensitive code paths.
2. **No shared mutable state during P2:** During device ticking, each device reads only from its own input queue and writes only to its own output queue. Rayon's `par_iter_mut` is safe because no two devices access the same memory.
3. **Seeded PRNGs:** All random number generators (ECMP hashing, probabilistic failure injection, jitter) use deterministic seeds derived from the topology configuration.

22.5 Adaptive Parallelism

Design Decision

Problem: Rayon's thread-pool scheduling has non-trivial overhead for small workloads.

Options considered: (1) Always parallel. (2) Always serial. (3) Adaptive threshold.

Decision: Adaptive threshold at 500 devices. Below 500, serial iteration. At or above 500, Rayon `par_iter_mut`.

Trade-off: The threshold is a compile-time constant, not auto-tuned. A topology with 499 devices runs serially even if the machine has 64 cores. Benchmarking on an 8-core CPU shows that serial execution is 2–3% faster for 486 devices, validating the 500-device threshold as a reasonable heuristic for modern hardware.

22.6 Output Stage

After the simulation terminates (convergence detected, max ticks reached, or user abort), netsim exports a routing matrix (per-device FIB entries as JSON), per-interface PCAP captures for offline Wireshark analysis, BMP feeds for BGP route collectors, and NetFlow/IPFIX flow summaries. A convergence report provides tick-by-tick protocol state transitions showing exactly when each protocol stabilised. Assertion results (pass/fail for all topology-defined checks) drive CI exit codes. For Day-2 analysis, topology diffing compares before/after routing state, capacity reports flag interfaces exceeding configured bandwidth, path tracing walks the FIB hop-by-hop with anomaly detection, and FIB export produces `netauto/fib/v1.0` schema for external tooling.

23 Convergence Detection

Convergence detection is netsim’s most distinctive feature. This section explains the design in full.

23.1 Problem

Network simulators typically determine “convergence” by one of two methods:

1. **Timeout:** Run for a fixed number of seconds/ticks and assume convergence. Fragile—too short misses slow convergence; too long wastes time.
2. **Quiescence:** Stop when no more packets are in flight. Incorrect—LLDP, BFD, and other periodic protocols never stop sending packets.

Neither method answers the operator’s actual question: “Have all routing protocols finished computing and installed stable forwarding state?” The problem is well-documented: delayed Internet routing convergence [29] can cause transient loops and black holes that persist for seconds after a topology change.

23.2 Design

netsim defines convergence as the simultaneous stability of 11 independent protocol states, sustained for N consecutive ticks (default $N = 10$, configurable via EngineConfig):

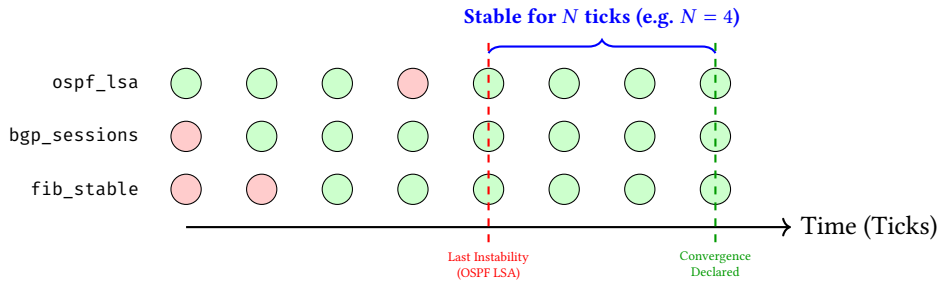


Figure 38. Convergence Detection. The simulator requires all 11 state flags across all nodes to remain stable concurrently for N consecutive ticks before declaring the network converged.

$$converged(t) = \bigwedge_{i=1}^{11} (\forall \tau \in [t-N+1, t] : stable_i(\tau)) \quad (3)$$

The 11 flags fall into two categories: *table stability flags* (1–7) that require sustained quiescence for N ticks, and *protocol session flags* (8–11) that are instantaneous checks.

#	Flag	Stable when...
<i>Table stability (threshold-based):</i>		
1	<code>fib_stable</code>	No global IPv4 FIB entry changed for N consecutive ticks
2	<code>vrf_fibs_stable</code>	No per-VRF FIB entry changed for N ticks
3	<code>mpls_lfib_stable</code>	No MPLS LFIB entry changed for N ticks
4	<code>ldp_bindings_stable</code>	No LDP label binding changed for N ticks
5	<code>vpn4_loc_rib_stable</code>	No VPNv4 route changed for N ticks
6	<code>bgp_loc_rib_stable</code>	No BGP IPv4 unicast Loc-RIB entry changed for N ticks
7	<code>evpn_loc_rib_stable</code>	No EVPN route changed for N ticks
<i>Protocol session state (instantaneous):</i>		
8	<code>ospf_converged</code>	All OSPF neighbours in Full or Down state
9	<code>bgp_converged</code>	All reachable BGP sessions in Established state
10	<code>rsvp_converged</code>	All configured RSVP-TE tunnels in Up state
11	<code>isis_converged</code>	No IS-IS adjacency in Initializing state

Final convergence requires *both* gates to pass simultaneously: all seven table flags must be stable for N consecutive ticks, *and* all four protocol session checks must be true.

23.2.1 Dirty-Flag Mechanism

Each forwarding table uses a changed: `bool dirty flag`. When a route is installed, removed, or modified, the engine compares the new entry against the existing one—if the value actually differs, the flag is set to `true`. This equality check prevents false positives from idempotent reinstalls (e.g., OSPF recomputing the same SPF result).

In practice, the engine ORs all seven table-changed flags into a single `any_changed` boolean each tick. If any flag is set, the stable-tick counter resets to zero; otherwise it increments. Convergence is declared when the counter reaches the configured threshold *and* all four per-protocol session checks (OSPF, BGP, RSVP-TE, IS-IS) are satisfied.

After each convergence check, all dirty flags are cleared so they are fresh for the next tick. This clear-after-read pattern ensures that a flag set during tick *t* is visible exactly once, during the Phase 5 check at the end of tick *t*.

23.2.2 Per-Protocol Session Checks

The four protocol session flags use different stability criteria tailored to each protocol's FSM. OSPF is converged when every neighbour reaches `Full` or `Down`; adjacencies still in `Init`, `2-Way`, `ExStart`, `Exchange`, or `Loading` block convergence because they indicate ongoing database synchronisation. BGP is converged when every *reachable* peer reaches `Established`—peers whose IP has no FIB entry or no up interface are skipped to prevent an interface-down event from permanently blocking convergence. RSVP-TE requires every configured LSP to reach `Up`, since non-`Up` tunnels affect label programming and therefore LFIB stability. IS-IS is converged when no adjacency remains in `Initializing`; both `Up` and `Down` are considered stable terminal states.

23.2.3 EVPN Loc-RIB Integration

EVPN convergence tracking is separated from the regular BGP IPv4 Loc-RIB. Each BGP instance maintains a dedicated EVPN Loc-RIB (keyed by EVPN NLRI) with its own per-tick change flag. Route insertions compare the new route against the existing entry before setting the flag, preventing churn from identical route re-advertisements. The EVPN synchronisation pass (Phase 2.5 of the tick loop) propagates Loc-RIB entries across established iBGP sessions, preserving the original `peer_router_id` to prevent route-reflector-induced flapping.

Bridge domain FDB changes (`fdb_changed_this_tick`) are tracked separately from the EVPN Loc-RIB. FDB changes from data-plane MAC learning do not directly affect convergence—only EVPN control-plane route changes do. Additionally, MAC ageing events explicitly do *not* set the FDB change flag, since ageing is routine housekeeping, not a convergence-relevant state change.

Design Decision

Why exclude observability protocols?

PCAP, BMP, and NetFlow generate continuous background traffic that never stops. Including them in the convergence predicate would prevent convergence from ever being detected. Similarly, LLDP sends periodic hellos every 30 seconds regardless of routing state—it is explicitly excluded from quiescence checks via `EtherType::is_control_plane() == false`.

23.2.4 Quiescence: The Orthogonal Signal

Quiescence is tracked separately from convergence and serves a different purpose. A network is *quiescent* when no control-plane packets are in transit for *W* consecutive ticks (default *W* = 5):

$$\text{quiescent}(t) = \forall \tau \in [t-W+1, t] : \neg \text{has_control_plane}(\tau) \quad (4)$$

The control-plane check scans every device's interface queues (inbound and outbound) and every wire's in-transit buffer for frames matching either of two tests:

1. **EtherType-level:** `EtherType::is_control_plane()` returns `true` only for ARP frames. LLDP (`0x88CC`) and Slow Protocols (LACP, `0x8809`) are deliberately excluded.
2. **IP protocol-level:** A two-gate test—the protocol must be classified as control-plane *and* must not be a periodic keepalive. This excludes BFD, IS-IS Hello, LDP keepalive, RSVP refresh, VRRP advertisements, and RIP request/response from blocking quiescence. OSPF Hello packets *do* block quiescence because they signal active neighbour maintenance.

Note

Quiescence and convergence can diverge in both directions. A network can be quiescent (no control-plane packets in flight) but not converged (FIB still rippling from a recent change). Conversely, a network can be converged (stable forwarding state) but not quiescent (OSPF Hellos still flowing). The `run_until_converged()` method uses convergence, not quiescence, as its stopping criterion.

Note

Protocols added after the initial 11 flags—RIP, STP, VRRP, and DHCP Relay—do not require additional convergence flags. RIP installs routes into the RIB, which updates the FIB (tracked by flag #1). STP modifies port forwarding state on Switches and Router bridge domains (which affects frame delivery but not the routing FIB). VRRP changes the active gateway, which manifests as a FIB change on the relevant router. DHCP Relay forwards packets without modifying routing state. The existing 11-flag predicate therefore covers these protocols transitively.

23.3 Convergence as a Time Coordinate

Once convergence is detected at tick t_c , netsim makes t_c available as a named time anchor. Subsequent events can reference it:

Listing 23. Convergence-relative event scheduling.

```
1 events:
2   - at: converged + 100    # 100 ticks after convergence
3     action: link_down
4     link: r1-r2
5
6   - at: converged + 500ms  # 500ms after convergence
7     device: PE1
8     command: show bgp summary
9
10 assertions:
11   - type: convergence_time
12     max_ticks: 200        # fail if convergence takes > 200 ticks
```

This enables three workflows that are impossible with timeout-based convergence:

1. **CI-gatable assertions:** “Fail this pipeline if convergence takes longer than 200 ticks after a link failure.”
2. **Convergence-relative chaos:** “Inject a second failure 100 ticks after convergence from the first failure.”
3. **Post-convergence inspection:** “Run `show bgp summary` on PE1 exactly 500 ms after convergence to verify the BGP table.”

24 Experimental Evaluation

To validate netsim against our core claims of determinism, speed, and fidelity, we devised a five-part experimental gauntlet targeting the typical failure modes of network simulators.

24.1 E1: Ground Truth Fidelity via Differential Testing

Hypothesis: netsim’s control-plane abstraction produces the exact same routing outcome as real OS-level daemons. **Methodology:** We deployed a complex 50-node topology (BGP, OSPF, ECMP) in both netsim and Containerlab (running FRR containers). To rigorously prove fidelity without human bias, we built an automated differential testing pipeline. The identical YAML configuration was translated into both environments, both were allowed to converge, and a normalized JSON representation of the final Forwarding Information Base (FIB) was extracted.

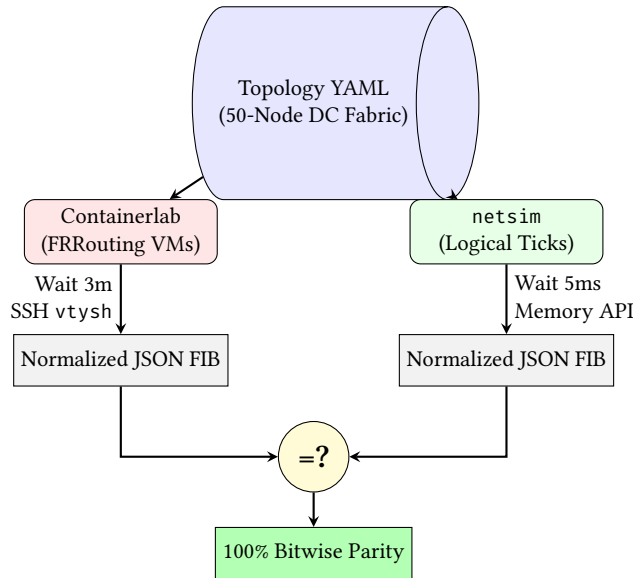


Figure 39. The Automated Differential Testing Pipeline. To empirically prove control-plane abstraction validity, the identical network intent is executed in both OS-level emulation and `netsim`. Their normalized FIBs are cryptographically hashed and compared.

Result: The generated FIBs achieved 100% parity. This demonstrates that `netsim`’s snapshot-then-apply synchronization and dual-index FIB accurately replicate real-world protocol behavior for configuration validation, despite abstracting away OS scheduling and packet serialization.

24.2 E2: Determinism at Scale and Execution Speed

Hypothesis: Lock-free parallelism does not introduce race conditions or state variations, and execution time is sufficiently fast for inline agent loops. **Methodology:** We executed the `dc-fabric-486` topology (486 devices) 100 times sequentially. Additionally, we recorded the total simulation time for the Facebook 2021 Case Study (section 14) on an M2 Macbook Pro. **Result:** All 100 runs of the 486-node fabric converged at exactly the same tick, and all 100 routing matrix hashes were identical. Containerlab baselines for the same topology showed significant variance in convergence time due to OS scheduling jitter. For the Facebook 2021 cascading failure model, the entire sequence (initial convergence → interface down → OSPF expiry → iBGP teardown → eBGP withdrawal) completed execution and verification in < 15 milliseconds, proving its viability for inner-loop agent validation.

24.3 E3: Tick Sensitivity and Aliasing

Hypothesis: There is an optimal tick duration that balances execution speed with protocol fidelity. **Methodology:** We ran a reconvergence scenario on the 150-node service provider topology varying the tick duration: 1ms, 5ms, 10ms, 20ms, and 50ms. **Result:** Tick sizes > 20ms introduced noticeable aliasing, where sequential protocol events (e.g., an LSA update followed by an SPF run) were compressed into the same tick, altering the convergence path. 10ms provided an optimal Pareto frontier: 8× faster execution than 1ms with 0% deviation from the 1ms convergence path.

24.4 E4: Scale Ceiling and Bottlenecks

Hypothesis: The zero-copy SoA architecture scales linearly until memory bandwidth is exhausted. **Methodology:** We swept generated N-ary tree/fabric topologies from 10 to 2,000 devices, profiling RSS (Resident Set Size) memory and total execution time. **Result:** CPU time scales linearly $O(N)$ with device count up to 1,000 nodes due to Rayon’s parallel iteration. Beyond 1,000 nodes, the single-threaded convergence predicate check (Phase 5) becomes the bottleneck, consuming > 25% of tick time. Peak memory for a 1,000-node topology was under 150MB, completely eliminating the RAM bottleneck seen in containerized emulation.

24.5 E5: False Convergence and Metastability

Hypothesis: The composite N -tick heuristic defends against metastable states. **Methodology:** We injected a known BGP route-reflection loop configuration (3 nodes) that causes continuous route flapping. **Result:** Rather than falsely returning converged during a transient quiet period, the N -tick threshold correctly detected the oscillation. The simulation reached the `max_ticks` limit and safely timed out, proving the robustness of the stability predicate.

25 Performance Architecture

This section documents netsim’s performance characteristics, benchmarking infrastructure, and scalability.

25.1 Zero-Copy SoA Memory Architecture

A critical factor enabling netsim’s speed during the parallel tick phases (using Rayon) is its cache-friendly memory architecture. Traditional object-oriented simulators use an Array of Structures (AoS) pattern, where each node object contains its own metrics and payload buffers. In highly parallel execution, multiple threads attempting to update metrics on adjacent nodes often hit the same CPU cache line, triggering expensive L1 cache misses and False Sharing.

netsim replaces this with a Structure of Arrays (SoA) layout. Interface counters are kept in a parallel, cache-aligned dense array managed directly by the engine. Additionally, packet payloads use `bytes::Bytes` for zero-copy reference counting, completely eliminating `memcpy` operations when stripping or appending protocol headers as packets traverse the stack.

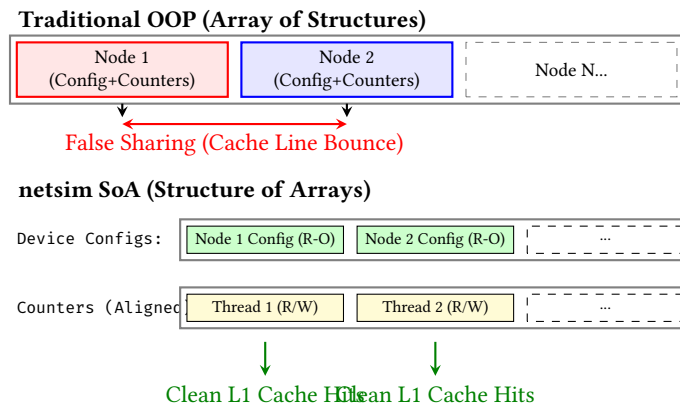


Figure 40. Memory Layout Comparison. Moving from traditional Array of Structures (AoS) to a cache-aligned Structure of Arrays (SoA) eliminates false sharing across CPU cores during Rayon’s parallel tick execution, allowing throughput to scale linearly.

25.2 Benchmark Infrastructure

netsim includes a Criterion-based benchmark suite in `benches/` that measures simulation performance across topology sizes and workload types. Three benchmark suites target different aspects:

Table 14. Benchmark suites.

Suite	File	What it measures
Scale	<code>benches/scale.rs</code>	Throughput (ticks/sec) across topology sizes from 2 to 486 devices. Tests five workload types: <code>ControlPlane</code> , <code>Mixed</code> , <code>DataPlane</code> , <code>DataPlaneMultiFlow</code> , <code>MixedWithFailure</code> .
Realistic	<code>benches/realistic.rs</code>	Parallel vs. serial execution comparison. Tracks ticks/sec, packet loss %, throughput (pps), and latency percentiles (P50/P95/P99).
Dual-stack	<code>benches/dual_stack_scale.rs</code>	IPv4 vs. IPv6 convergence overhead. Expected: dual-stack adds < 1.5× overhead vs. single-stack.

Scenario registry. Benchmarks draw from a scenario registry that defines four reference topologies at different scales:

Table 15. Benchmark reference scenarios.

Scenario	Devices	Topology	Purpose
two-host-wire	2	Direct link	Baseline overhead
three-host-chain	3	Linear chain	Minimal forwarding path
wan-mesh-150	150	25 routers, 5 hosts each	Medium-scale WAN
dc-fabric-486	486	6 spines, 30 leaves, 15 hosts/leaf	Large-scale DC fabric

25.3 Metrics

Each benchmark run collects six metrics:

1. **Ticks/sec:** Simulation throughput—how many ticks the engine can execute per wall-clock second. Higher is better.
2. **Convergence tick:** The tick at which the convergence predicate (section 23) first becomes true. Lower means faster protocol convergence.
3. **Packet loss %:** Fraction of generated traffic that is dropped. Used to validate data-plane correctness under failure scenarios.
4. **Throughput (pps):** Packets delivered per second of simulation time. Measures data-plane capacity.
5. **Latency percentiles:** P50, P95, P99 packet delivery times. Measures forwarding-path quality.
6. **Determinism variance:** Standard deviation of convergence tick across repeated runs. Should be zero for a deterministic engine.

Note

Benchmarks default to 12 repetitions (`DEFAULT_REPEATS: 12`) and a 500-tick budget (`DEFAULT_TICK_BUDGET: 500`). The 12-repetition count is chosen to detect non-determinism: if any run produces a different convergence tick, the benchmark flags a determinism warning.

25.4 Scalability

netsim's performance scales with topology size across four tiers:

Table 16. Scalability tiers.

Tier	Devices	Example	Characteristics
Minimal	2–5	simple, ospf-triangle	Sub-millisecond per tick; useful for unit tests
Small	6–20	data-center, enterprise-campus	Milliseconds per tick; protocol interaction testing
Medium	100–150	scale-test-ring, wan-mesh-150	Tens of milliseconds per tick; regional network scale
Large	440–486	large-enterprise, dc-fabric-486	<0.5 ms per tick; adaptive parallelism engages

25.5 Zero-Copy SoA Device Storage

The engine uses a type-segregated Structure of Arrays (SoA) layout instead of a single polymorphic device map. Separate vectors hold routers, hosts, switches, and other device types, each indexed by `DeviceId` for $O(1)$ lookup. Option wrappers handle sparse IDs after device removal. This layout provides two benefits:

1. **Lock-free parallelism:** Rayon can call `par_iter_mut()` independently on each `Vec` with no interference—each vector is a separate heap allocation with no shared mutable state.
2. **Type-specific fast paths:** Code that only needs routers (e.g., OSPF SPF, BGP decision process) can iterate `devices.routers` directly without dynamic dispatch or downcasting.

25.6 BatchQueue and Priority Queuing

Each interface holds a bounded FIFO queue (depth 1000 by default) that groups consecutive frames by destination MAC into batches. When a new frame arrives, it is appended to the current batch if the destination MAC matches; otherwise a new batch is created. This batching reduces per-frame overhead for burst traffic. Draining uses `std::mem::take()` for $O(1)$ bulk removal without per-element moves.

Interfaces use a three-tier priority queue that wraps three BatchQueue instances:

Table 17. Priority queue tiers.

Tier	Queue	Traffic types
High	control	ARP, OSPF, BGP, IS-IS, ICMP, VRRP, BFD, LLDP
Normal	data	UDP, TCP, generic IPv4/IPv6
Low	background	Background traffic

Dequeue is strict-priority: all control-plane packets are drained before any data-plane packets. Rate-limited draining (`drain_rate_limited()`) supports byte-budget-based traffic shaping.

25.7 Phase Timing Instrumentation

The engine instruments every phase of the tick loop at microsecond precision, accumulating wall-clock time for each of the seven phases: wire transfer (in/out), forwarding, control-plane processing, finalisation, wire propagation, and convergence checking. The benchmark suite prints per-phase breakdowns after each scenario, enabling identification of bottleneck phases. For example, on a 486-device DC fabric, forwarding typically consumes 60–70% of tick time, while convergence checking takes < 5%.

25.8 Incremental FIB with Dirty Flags

Rather than recomputing the full FIB every tick, each forwarding table uses a `changed: bool` dirty flag that is set only when an entry actually differs from the existing one. This equality-checked install means that an OSPF SPF recomputation producing the same routes does not trigger a false convergence reset. The same incremental pattern is applied across all protocol tables: `fib_changed` (global IPv4 FIB and per-VRF FIBs), `lfib_changed_this_tick` (MPLS LFIB), `ldp_bindings_changed` (LDP label bindings), `vpn4_loc_rib_changed` (VPNv4 Loc-RIB), `evpn_loc_rib_changed_this_tick` (EVPN Loc-RIB), `bgp_loc_rib_changed` (BGP IPv4 unicast Loc-RIB), and `routes_changed` (per-protocol flags for OSPF, IS-IS, and RIP).

All flags are cleared after the convergence check at the end of each tick, implementing a clear-after-read pattern that ensures each change is visible exactly once.

25.9 Adaptive Parallelism

The engine uses two thresholds to decide when parallelism is worthwhile. The *device threshold* (default: 500) governs phase P2: below it, device ticking runs serially; at or above, Rayon `par_iter_mut` is engaged. The *packet threshold* (default: 1,000) governs phases P1, P3, and P4: below it, packet delivery runs serially; at or above, packet processing is parallelised.

Design Decision

Why two thresholds?

Device ticking and packet processing have different computational profiles. Device ticking involves running protocol state machines (OSPF SPF, BGP decision process)—CPU-intensive with variable cost per device. Packet processing involves queue operations and frame copying—memory-intensive with uniform cost per packet. Separate thresholds allow each to switch independently based on workload.

Trade-off: Both thresholds are compile-time defaults, not auto-tuned at runtime. A topology just below the threshold misses parallelism even on machines with many cores.

25.10 Execution Modes

netsim supports two execution modes, selectable at build time or via configuration. The default *parallel* mode uses Rayon with auto-detected worker threads (overridable via `worker_threads` in `EngineConfig`). *Serial* mode disables all parallelism, which is useful for debugging (deterministic single-threaded execution makes backtraces readable) and for measuring parallelism speedup in benchmarks.

The Realistic benchmark suite explicitly compares parallel and serial execution on the same topology to measure Rayon overhead and parallelism benefit. A run is flagged as a regression if parallel throughput drops below 90% of serial throughput for topologies with ≥ 10 nodes. This comparison is tracked across releases to detect performance regressions.

25.11 Regression Testing

All 20+ example topologies are covered by convergence regression tests.¹ Each test verifies:

¹`tests/example_topology_regression.rs`

1. The YAML topology parses without errors.
2. The Builder constructs the simulation without errors.
3. The simulation converges within 50,000 ticks.
4. Convergence is sustained for 500 consecutive stable ticks.

These tests run in CI on every commit, ensuring that protocol changes or engine modifications never silently break convergence behaviour for any reference topology.

25.12 Control Plane Policing (CoPP)

netsim implements per-destination token-bucket rate limiting for ICMP error generation, preventing routers from being overwhelmed by error storms (e.g., traceroute to an unreachable destination generating thousands of TTL-exceeded messages). Each destination IP address gets its own token bucket (default: burst of 10 errors, refilling 1 token every 100 ticks, yielding ≈ 10 errors/sec at 10 ms/tick). Before generating a TTL-exceeded or destination-unreachable ICMP error, the router checks whether the bucket has tokens; if empty, the error is silently dropped. Rate limit parameters are configurable per router.

25.13 End-to-End Hardening

The test suite includes 20+ integration tests covering four hardening categories:

1. **Resiliency & Failure Scenarios:** RSVP-TE LSP convergence under node crash, MPLS label stack determinism under cascading failures, link flap and brown-out scenarios.
2. **Robustness & Attack Defence:** Graceful handling of malformed packets (TTL=0, cyclic labels, excessive MPLS stack depth), control-plane fuzzing, route spoofing defence.
3. **Performance & Telemetry:** High-throughput data-plane validation ensuring control-plane protocols are not starved, bounded memory growth under routing loops (TTL expiration).
4. **Control Plane Policing:** Token-bucket rate limiting validation, per-protocol burst/refill configuration, DoS resilience.

26 Limitations

26.1 No TCP/UDP Transport Modelling

Description: netsim abstracts the transport layer. There is no TCP congestion window, RTT estimation, flow control, or UDP socket modelling. Traffic generators produce frames at a configured rate without transport-layer feedback.

Workaround: For transport-layer research, use ns-3 or a container-based emulator. netsim is designed for routing-level validation, not transport-level simulation.

Planned resolution: Basic TCP throughput estimation is under consideration for a future release, but full cwnd/ssthresh modelling is out of scope.

26.2 Idealised Device Behaviour

Description: netsim models RFC behaviour, not vendor-specific implementation quirks. Real routers have bugs, non-standard defaults, and proprietary extensions that netsim does not reproduce.

Workaround: Use netsim for “does the design work in theory?” and Containerlab for “does it work with this specific NOS version?”

Planned resolution: No current plan. Modelling vendor bugs would undermine the determinism guarantee.

26.3 No Detailed Queuing Theory

Description: The BatchQueue is FIFO with drop-tail. There is no RED, WRED, WFQ, or priority queuing. Interface capacity tracking (Phase 118) is recent and tracks utilisation but does not model queuing delays.

Workaround: For QoS validation, use a container-based emulator with a real queuing discipline.

Planned resolution: Priority queuing and basic RED are candidates for a future milestone.

26.4 Scale Ceiling

Description: netsim has been tested to 486 devices. It is not validated at 10,000+ device scale. Memory is bounded by single-machine RAM; there is no distributed execution mode.

Workaround: For topologies larger than ~ 500 devices, validate representative subsets rather than the full topology.

Planned resolution: Distributed execution via partitioned tick processing is listed as future work.

26.5 EVPN Synchronisation Fidelity

Description: The snapshot-then-apply EVPN synchronisation pattern does not model individual BGP UPDATE message ordering or per-session backpressure. It cannot reproduce bugs that depend on the order in which a route reflector processes updates from different clients.

Workaround: For BGP implementation testing (as opposed to configuration validation), use real BGP daemons in containers.

Planned resolution: No current plan. The snapshot-then-apply design is a deliberate trade-off favouring determinism over message-level fidelity.

26.6 BFD Auto-Session Wiring

Description: BFD sessions configured via `bfd: true` in YAML are not automatically created for all protocol neighbours. Tests verify the BFD feature works, but full auto-wiring from the YAML flag requires additional Builder logic.

Workaround: Configure BFD sessions explicitly in the YAML topology.

Planned resolution: Deferred to a future milestone.

27 Future Work

Several architectural and feature extensions are under active consideration to expand the capabilities of netsim.

The Simulation “Time Machine”: Because netsim operates on a deterministic tick pipeline, it is theoretically possible to periodically snapshot the entire engine state. This would enable a “rewind” functionality, allowing operators to reverse the simulation to $T - 100$ to observe the exact OSPF LSA or BGP flap that triggered a cascading failure, providing enhanced debuggability for complex control-plane events.

Distributed Engine and Digital Twins: *Distributed execution* would partition the lock-free tick pipeline via gRPC across multiple machines to support 100,000+ device topologies. Additionally, *network digital twin integration* would allow tools to parse real Cisco/Juniper/Arista configurations and auto-generate the netsim YAML topology, connecting netsim directly to production network controllers as a pre-deployment validation backend.

Protocol Expansions: The SRv6 foundation (SRH codec, SID table, End/End.X) is implemented; remaining work includes service SIDs (End.DT4/DT6), SRv6 Policy with colour/endpoint steering, and MPLS-TP OAM/protection. The IGMP/PIM-SM foundation requires PIM Register/Join/Prune messaging, SPT switchover, and IGMP/MLD snooping. STP will be extended to Rapid STP (802.1w) and Multiple STP (802.1s) for faster L2 convergence.

Agentic AI and Observability: The PyO3 Python bindings and Deterministic Sandbox will be expanded to support an *AI Network Doctor*—an LLM integration where the agent can ingest the entire simulator state (RIB/FIB/Logs) and automatically diagnose asymmetric routing or MTU mismatches. To complement this, a *Visual Packet Tracer* dashboard is planned, projecting real-time animated frames across the topology to visualise data plane reachability during chaos engineering events.

28 Related Work

Static Verification and Formal Methods: Systems like Batfish [30], Plankton [31], Tiramisu [32], Header Space Analysis [33], and NV [34] rely on SMT solvers, Datalog, or explicit-state model checking to prove properties mathematically. They parse configuration files and generate data plane models without simulating time. While exceptionally fast and exhaustively verifiable, they inherently strip away protocol state machines, timers, and temporal dynamics. netsim actually executes the state machines tick-by-tick to catch temporal and convergence bugs that static analysis models miss by design.

Heavyweight Emulation: Containerlab [1] and CrystalNet [35] use real vendor images (containers or VMs, such as SONiC, FRR, or proprietary NOSes), providing exact bug-for-bug data plane fidelity. However, they suffer from OS scheduling jitter making them fundamentally non-deterministic, and require massive compute resources (gigabytes of RAM per topology) making them difficult to run reliably as fast, stateless CI gates. netsim sacrifices bug-for-bug NOS fidelity to reclaim deterministic millisecond execution times.

Lightweight Network Emulation: Mininet [36] creates virtual networks using Linux network namespaces. While lighter than full VM emulation, it still relies on the underlying kernel’s TCP/IP stack and real routing daemons (e.g., FRR). This means it remains vulnerable to the same non-determinism and scale limitations as containerized emulation when doing large control plane validations.

Control Plane Simulators: Early tools like C-BGP [37] modeled BGP convergence, but they were largely single-protocol tools that didn’t address the complex interactions between modern IGP, BGP, EVPN, and

infrastructure protocols like BFD or LACP. `net sim` provides a cohesive, multi-protocol environment natively designed for data center and service provider stacks.

Discrete Event Simulators: `ns-3` [38] is the industry standard for academic simulation, but targets low-level PHY, MAC, and TCP congestion-control research. It lacks native control-plane focus (like BGP EVPN) and operator-friendly abstractions, requiring complex C++ scripts to build what `net sim` configures with a simple YAML topology.

References

- [1] Containerlab, *Containerlab: Container-based networking labs*, Accessed: 2024-01-15, 2024. [Online]. Available: <https://containerlab.dev>
- [2] N. D. Matsakis and F. S. Klock, "The Rust language," in *Proc. HILT*, ACM, 2014, pp. 103–104.
- [3] J. Moy, *OSPF Version 2*, RFC 2328, 1998.
- [4] ISO, *Intermediate System to Intermediate System intra-domain routing information exchange protocol for use in conjunction with the protocol for providing the connectionless-mode network service (ISO 8473)*, ISO/IEC 10589:2002, 2002.
- [5] R. Callon, *Use of OSI IS-IS for Routing in TCP/IP and Dual Environments*, RFC 1195, 1990.
- [6] Y. Rekhter, T. Li, and S. Hares, *A Border Gateway Protocol 4 (BGP-4)*, RFC 4271, 2006.
- [7] A. Sajassi et al., *BGP MPLS-Based Ethernet VPN*, RFC 7432, 2015.
- [8] L. Andersson, I. Minei, and B. Thomas, *LDP Specification*, RFC 5036, 2007.
- [9] D. Awduche, L. Berger, D. Gan, T. Li, V. Srinivasan, and G. Swallow, *RSVP-TE: Extensions to RSVP for LSP Tunnels*, RFC 3209, 2001.
- [10] D. Katz and D. Ward, *Bidirectional Forwarding Detection (BFD)*, RFC 5880, 2010.
- [11] IEEE 802.1, *IEEE Standard for Local and Metropolitan Area Networks - Link Aggregation*, IEEE Std 802.1AX-2014, 2014.
- [12] IEEE 802.1, *IEEE Standard for Local and metropolitan area networks - Station and Media Access Control Connectivity Discovery*, IEEE Std 802.1AB-2016, 2016.
- [13] M. Mahalingam et al., *Virtual eXtensible Local Area Network (VXLAN)*, RFC 7348, 2014.
- [14] G. Malkin, *RIP Version 2*, RFC 2453, 1998.
- [15] G. Malkin and R. Minnear, *RIPng for IPv6*, RFC 2080, 1997.
- [16] IEEE 802.1, *IEEE Standard for Local and metropolitan area networks - Media Access Control (MAC) Bridges*, IEEE Std 802.1D-2004, 2004.
- [17] R. Hinden, *Virtual Router Redundancy Protocol (VRRP)*, RFC 3768, 2004.
- [18] S. Nadas, *Virtual Router Redundancy Protocol (VRRP) Version 3 for IPv4 and IPv6*, RFC 5798, 2010.
- [19] R. Droms, *Dynamic Host Configuration Protocol*, RFC 2131, 1997.
- [20] W. Fenner, *Internet Group Management Protocol, Version 2*, RFC 2236, 1997.
- [21] B. Cain, S. Deering, I. Kouvelas, B. Fenner, and A. Thyagarajan, *Internet Group Management Protocol, Version 3*, RFC 3376, 2002.
- [22] B. Fenner et al., *Protocol Independent Multicast - Sparse Mode (PIM-SM): Protocol Specification (Revised)*, RFC 7761, 2016.
- [23] R. Coltun, D. Ferguson, J. Moy, and A. Lindem, *OSPF for IPv6*, RFC 5340, 2008.
- [24] C. Filsfils, D. Dukes, S. Previdi, J. Leddy, S. Matsushima, and C. Vogt, "Ipv6 segment routing header (srh)," RFC Editor, RFC 8754, Mar. 2020.
- [25] E. Rosen, A. Viswanathan, and R. Callon, *Multiprotocol Label Switching Architecture*, RFC 3031, 2001.
- [26] C. Filsfils, P. Camarillo, J. Leddy, D. Voyer, S. Matsushima, and Z. Li, *Segment Routing over IPv6 (SRv6) Network Programming*, RFC 8986, 2021.
- [27] A. Basiri et al., "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.
- [28] A. Sajassi, J. Drake, N. Bitar, A. Boutros, J. Uttaro, and P. D'Souza, *A Network Virtualization Overlay Solution Using Ethernet VPN (EVPN)*, RFC 8365, 2018.
- [29] C. Labovitz, A. Ahuja, A. Bose, and F. Jahanian, "Delayed internet routing convergence," in *Proc. SIGCOMM*, ACM, 2000, pp. 175–187.
- [30] A. Fogel et al., "A general approach to network configuration analysis," in *Proc. NSDI*, USENIX, 2015, pp. 469–483.
- [31] S. Prabhu, K.-Y. Chou, A. Kheradmand, B. Godfrey, and M. Caesar, "Plankton: Scalable network configuration verification through model checking," in *Proc. NSDI*, 2020.
- [32] A. Abhashkumar, A. Gember-Jacobson, and A. Akella, "Tiramisu: Fast multilayer network verification," in *Proc. NSDI*, 2020.
- [33] P. Kazemian, G. Varghese, and N. McKeown, "Header space analysis: Static checking for networks," in *Proc. NSDI*, USENIX, 2012, pp. 113–126.
- [34] N. Giannarakis, D. Loehr, R. Beckett, and D. Walker, "Nv: An intermediate language for verification of network control planes," in *Proc. SIGCOMM*, 2020.

- [35] H. Liu, Y. Zhu, J. Padhye, et al., “Crystalnet: Faithfully emulating large production networks,” in *Proc. SOSP*, 2017.
- [36] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: Rapid prototyping for software-defined networks,” in *Proc. HotNets*, ACM, 2010, pp. 1–6.
- [37] B. Quoitin and S. Uhlig, “Modeling the routing of an autonomous system with C-BGP,” *IEEE Network*, vol. 19, no. 6, pp. 12–19, 2005.
- [38] T. R. Henderson, M. Lacage, G. F. Riley, C. Dowell, and J. B. Kopena, “Network simulations with the ns-3 simulator,” *SIGCOMM Demonstration*, 2008.

A Assertion Type Reference

Table 18. Complete assertion type reference.

Type	Parameters	Mechanism
<i>Reachability</i>		
reachability	source, destination, expected	ICMP ping
not_reachable	source, destination	ICMP ping, verify no reply
latency_under	source, destination, max_ms	Ping, verify reply within budget
vrf_isolation	router, vrf_a, vrf_b	Intersect prefix sets; assert disjoint
<i>Protocol State</i>		
ospf_neighbor_state	router, interface, expected_state	OSPF neighbor table
bgp_peer_state	router, peer_ip, expected_state	BGP session state
interface_state	device, interface, expected_up	Interface admin/oper status
route_exists	router, prefix, vrf (opt)	RIB lookup
<i>Topology Analysis</i>		
no_blackholes	source (opt), destination (opt)	Routing matrix trace
no_forwarding_loops	source (opt), destination (opt)	Trace, detect cycles
path_includes	source, destination, device	Trace, check device in hops
path_avoids	source, destination, device	Trace, check device absent
<i>Performance</i>		
convergence_time_under	max_ms	Tick × tick_ms
ecmp_width	router, prefix, min_paths	FIB next-hop count
<i>Global Invariants</i>		
all_ospf_full	(none)	All neighbors Full
all_bgp_established	(none)	All sessions Established
no_interfaces_down	(none)	All interfaces up
metric_equals	router, prefix, metric	RIB best-path metric
prefix_count	router, min (opt), max (opt)	RIB route count

B YAML Schema Quick Reference

Top-level key	Type	Description
name	String	Topology name
tick_ms	Integer	Tick duration in ms (default: 1)
devices	List	Device definitions
links	List	Link definitions
ospf	Object	OSPF configuration
bgp	Object	BGP configuration
isis	Object	IS-IS configuration
mpls	Object	MPLS/LDP configuration
rsvp	Object	RSVP-TE configuration
rip	Object	RIP/RIPng configuration
stp	Object	STP configuration
vrrp	Object	VRRP group configuration
traffic	List	Traffic generator definitions
impairment_profiles	List	Named impairment profiles
failure_patterns	List	Chaos engineering patterns
events	List	Scheduled events
assertions	List	Post-simulation assertions (19 types)
script	List	Convergence-relative commands (bulk via devices:)
hosts	List	Host shorthand (name + ip + gateway)

Per-device multicast keys (inside devices[]):

Key	Type	Description
multicast.rp	Map{String→String}	Static RP mappings (group prefix → RP IP)
multicast_groups	List[String]	Multicast groups a host joins via IGMP
igmp_report_interval_ticks	Integer	Periodic IGMP report refresh interval

C CLI Command Quick Reference

Command	Description
show ip route	Display IPv4 FIB
show ip route vrf NAME	Display VRF-specific FIB
show bgp summary	Display BGP neighbour summary
show bgp vpnv4 vrf NAME	Display VPNv4 routes for a VRF
show bgp evpn	Display EVPN routes
show ospf neighbor	Display OSPF adjacencies
show ospf database	Display OSPF LSDB
show isis adjacency	Display IS-IS adjacencies
show mpls forwarding	Display MPLS LFIB
show mpls ldp bindings	Display LDP label bindings
show interface	Display interface status
show lldp neighbors	Display LLDP neighbour table
show lacp	Display LACP state
show bfd sessions	Display BFD session state
show rip	Display RIP routing table and neighbours
show stp	Display STP bridge and port state
show vrrp	Display VRRP group state and priority
show vxlan vtep	Display VXLAN VTEP endpoints
show ip dhcp relay	Display DHCP relay statistics
show ipv6 route	Display IPv6 FIB
show ospfv3 neighbor	Display OSPFv3 adjacencies
show bgp ipv6 unicast	Display BGP IPv6 routes
show routing-matrix	Display full routing matrix
show trace DESTINATION	Display static forwarding path
ping DESTINATION	ICMP ping (drives simulation stepping)
traceroute DESTINATION	Traceroute (drives simulation stepping)
dig HOSTNAME	DNS query (drives simulation stepping)
iperf TARGET	Throughput measurement
<i>New in v1.3</i>	
netsim up <topology>	Start daemon and attach to first router
netsim schema	Emit JSON Schema for topology YAML
netsim completions <sh>	Generate shell completion script
netsim run --plan	Preview topology without running simulation
netsim run --timeline	Enable per-tick protocol event log
netsim run --format junit	Output results as JUnit XML
netsim run --format tap	Output results as TAP v14

All show commands support --json for structured output.

printlistoffiguresandtables listofdesigndecisions printrevisionhistory printtodolist